

beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is a solution to this [latex.org forum query](#).

Contents

1 Installation

1.1 Package manager

When not already available, **beanoves** package may be installed using a TEX distribution's package manager, either from the graphical user interface, or with the relevant command (`tlmgr` for **T_EX** Live and `mpm` for **MiK_TE_X**). This should install files **beanoves.sty** and its debug version **beanoves-debug.sty** as well as **beanoves-doc.pdf** documentation.

1.2 Manual installation

The **beanoves** source files are available from the [source repository](#). They can also be fetched from the [CTAN repository](#).

1.3 Usage

The **beanoves** package is imported by putting `\RequirePackage{beanoves}` in the preamble of a **L^AT_EX** document that uses the **beamer** class. Should the package cause problems, its features can be temporarily deactivated with simple commands `\BeanovesOff` and `\BeanovesOn`.

2 Minimal example

The L^AT_EX document below is a contrived example to show how the **beamer** overlay specifications have been extended. More demonstration files are available from the [beanoves source repository](#).

```
1 \documentclass{beamer}
2 \RequirePackage{beanoves}
3 \begin{document}
4 \Beanoves {
5   A = 1:4,
6   B = A.last::3,
7   C = B.next,
8 }
9 \begin{frame}
10  {\Large Frame \insertframenumber}
11  {\Large Slide \insertslidenumber}
12 -- \visible<?(A.1)> {Only on slide 1}\\
13 -- \visible<?(B.range)> {Only on slides 4 to 6}\\
14 -- \visible<?(C.1)> {Only on slide 7}\\
15 -- \visible<?(A.2)> {Only on slide 2}\\
16 -- \visible<?(B.2:B.last)> {Only on slides 5 to 6}\\
17 -- \visible<?(C.2)> {Only on slide 8}\\
18 -- \visible<?(A.next)-> {From slide 5}\\
19 -- \visible<?(B.3:B.last)> {Only on slide 6}\\
20 -- \visible<?(C.3)> {Only on slide 9}\\
21 \end{frame}
22 \end{document}
```

On line 4, we use the `\Beanoves` command to declare *named overlay sets*. On line 5, we declare an overlay set named ‘A’, which is a range starting at slide 1 and ending at slide 4. On line 12, the extended *named overlay specification* `?(A.1)` stands for 1 because 1 is the first index of the overlay set named A. On line 15, `?(A.2)` stands for 2 whereas on line 18, `?(A.next)` stands for 5. On line 6, we declare a second overlay set named ‘B’, starting after the 3 slides of ‘A’ namely 4. Its length is 3 meaning that its last slide number is 6, thus each `?(B.last)` is replaced by 6. The next slide number after slide range ‘B’ is 7 which is also the start of the third slide range due to line 7.

3 Named overlay sets

3.1 Presentation

Within a **beamer** frame, there are different slides that appear in turn according to overlay specifications. The main overlay set is a range of integers covering all the slide numbers, from one to the total amount of slides. In general, an overlay set is a range of positive integers identified by a unique name. The main practical interest is that such sets may be defined relative to one another, we can even have lists of overlay sets. Finally, we can use these lists to build and organize **beamer** overlay specifications logically.

3.2 Named overlay reference

A.1, C.2 are *named overlay references*, as well as A and X!A.B.C.2. More precisely, they are string identifiers, each one referencing a well defined static integer or list of ranges to be used in beamer overlay specifications. They have 2 components:

1. the *frame identifier* $\langle id \rangle$ before the exclamation mark !, like X, in X!A.B.C.2, optional
2. the *path* $\langle c_1 \rangle \dots \langle c_j \rangle$ like A.B.C, required ($j > 0$).

The *full path* is $\langle id \rangle ! \langle c_1 \rangle \dots \langle c_j \rangle$. The *frame ids* and path components $\langle c \rangle$'s are alphanumeric case sensitive identifiers, with possible underscores but with no space. Unicode symbols above U+00A0 are allowed if the underlying T_EX engine supports it. Only the *frame id* is allowed to be empty, in which case it may apply to any common frame. The *path components* must not consist of only lowercase letters¹.

The mapping from *named overlay references* to sets of integers is defined at the global T_EX level to allow its use in `\begin{frame}<...>` and to share the same overlay sets between different frames. Hence the *frame id* due to the need to possibly target a particular frame.

3.3 Defining named overlay sets

In order to define *named overlay sets*, we can either execute the next `\Beanoves` command before a beamer frame environment, or use the custom `beanoves` option of this environment.

`\Beanoves` `\Beanoves{\langle ref_1 \rangle = \langle spec_1 \rangle, \dots, \langle ref_j \rangle = \langle spec_j \rangle}`

`\Beanoves*` Each $\langle ref \rangle$ key is a *named overlay reference* whereas each $\langle spec \rangle$ is an *overlay set specifier*. When the same $\langle ref \rangle$ key is used multiple times, only the last one is taken into account.

When performed at the document level, the `\Beanoves` command starts by cleaning what was set by previous calls. When performed inside L^AT_EX environments, each new call cumulates with the previous one. Notice that the argument of this function can contain macros: they will be exhaustively expanded at resolution time².

`beanoves` `beanoves = {\langle ref_1 \rangle = \langle spec_1 \rangle, \dots, \langle ref_j \rangle = \langle spec_j \rangle}`

The `\Beanoves` arguments take precedence over both the `\Beanoves*` arguments and the `beanoves` options. This allows to provide an overlay name only when not already defined, which is helpfull when the very same frame source is included multiple times in different contexts. Notice that $\langle ref \rangle = 1$ can be shortened to $\langle ref \rangle$.

3.3.1 Value specifiers

Hereafter $\langle value \rangle$ denotes a numerical expression.

¹This will avoid collisions with the `fp` module of `expl3`.

²Precision is needed about the exact time when the expansion occurs.

Standalone

$\langle \text{ref} \rangle = \langle \text{value} \rangle$, a *value specifier* for a single number. When $= \langle \text{value} \rangle$ is omitted it defaults to $=1$.

The numerical expressions are evaluated and then rounded using `\fp_eval:n`. They can contain mathematical functions and *named overlay references* defined above. If this reference concerns a set of values, only the first one is considered.

The corresponding overlay set can be seen as a *value counter*.

3.3.2 Colon delimited range specifiers

In beamer, integer ranges are denoted by $\langle \text{first} \rangle - \langle \text{last} \rangle$. In order to authorize algebraic expressions for $\langle \text{first} \rangle$ and $\langle \text{last} \rangle$, the dash character $-$ is replaced by a colon $:$ in beanoes. Hereafter $\langle \text{first} \rangle$, $\langle \text{last} \rangle$ and $\langle \text{length} \rangle$ are placeholders for *value specifiers*.

Standalone

$\langle \text{ref} \rangle = \langle \text{first} \rangle :$,
 $\langle \text{ref} \rangle = \langle \text{first} \rangle ::$, for the infinite range of signed integers starting at and including $\langle \text{first} \rangle$.

$\langle \text{ref} \rangle = \langle \text{first} \rangle : \langle \text{last} \rangle$,
 $\langle \text{ref} \rangle = \langle \text{first} \rangle :: \langle \text{length} \rangle$,
 $\langle \text{ref} \rangle = : \langle \text{last} \rangle :: \langle \text{length} \rangle$,
 $\langle \text{ref} \rangle = :: \langle \text{length} \rangle : \langle \text{last} \rangle$, are variants for the same finite range of signed integers starting at and including $\langle \text{first} \rangle$, ending at and including $\langle \text{last} \rangle$, provided $\langle \text{first} \rangle + \langle \text{length} \rangle = \langle \text{last} \rangle + 1$. $\langle \text{first} \rangle$ can be omitted, in which case it defaults to 1. Additionally $: \langle \text{last} \rangle$ and $:: \langle \text{length} \rangle$ are then equivalent.

$\langle \text{ref} \rangle = : \langle \text{last} \rangle$,
 $\langle \text{ref} \rangle = :: \langle \text{length} \rangle$, are syntactic sugar when $\langle \text{first} \rangle$ is 1.

3.3.3 List specifiers

$\langle \text{ref} \rangle = [\langle \text{def}_1 \rangle, \dots, \langle \text{def}_j \rangle]$, where $\langle \text{def}_k \rangle$, $1 \leq k \leq j$, is one of

- $\langle \text{index} \rangle = \langle \text{value} \rangle$,
- $\langle \text{value} \rangle$, a shortcut for $\langle i \rangle = \langle \text{value} \rangle$, $\langle i \rangle$ being the smallest positive integer such that $\langle \text{ref} \rangle . \langle i \rangle$ is not already defined.

The first step is to remove previous $\langle \text{ref} \rangle$ related definitions, then execute the various $\langle \text{ref} \rangle . \langle i \rangle = \langle \text{value} \rangle$ definitions in the order given. Here is the **implementation**.

$\langle \text{ref} \rangle = \{ \langle \text{def}_1 \rangle, \dots, \langle \text{def}_j \rangle \}$, where $\langle \text{def}_k \rangle$, $1 \leq k \leq j$, is one of

- $\langle \text{name} \rangle = \langle \text{spec} \rangle$,
- $\langle \text{name} \rangle$ for $\langle \text{name} \rangle = 1$,

The first step is to remove previous $\langle \text{ref} \rangle$ related definitions, then execute the various $\langle \text{ref} \rangle . \langle \text{name} \rangle = \langle \text{spec} \rangle$ definitions in the order given. In a final step, the $\langle \text{name} \rangle$'s are collected in a comma sepated list to initialize $\langle \text{ref} \rangle$ with. $\langle \text{spec} \rangle$ is any specifier. Here is the **implementation**.

4 Resolution of $\langle \dots \rangle$ query expressions

This is the key feature of the `beanoves` package, extending `beamer overlay specifications` normally included between pointed brackets. Before the *overlay specifications* are processed by the `beamer` class, the `beanoves` package scans them for any occurrence of $\langle \langle \text{queries} \rangle \rangle$. Each one is then evaluated and replaced by its resolved static counterpart. The overall result is finally forwarded to the `beamer` class.

The $\langle \text{queries} \rangle$ argument is a comma separated list of individual $\langle \text{query} \rangle$'s processed from left to right as explained below. Notice that nesting a $\langle \dots \rangle$ query expression inside another query expression is supported, embedded expressions being resolved first.

The named overlay sets defined above are queried for integer numerical values that will be passed to `beamer`. Turning an *overlay query* into the static expression it represents, as when above $\langle A.1 \rangle$ was replaced by 1, is denoted by *overlay query resolution* or simply *resolution*. The process starts by replacing any *query reference* by its value as explained below until obtaining numerical expressions that are evaluated and finally rounded to the nearest integer to feed `beamer` with either ranges or numbers. When the *query reference* is a previously declared $\langle \text{ref} \rangle$, like **A** after **A=1**, it is simply replaced by the corresponding declared $\langle \text{value} \rangle$, here 1. Otherwise, we use *implicit overlay queries* and their *resolution rules* depending on the definition of the named overlay set. Hereafter $\langle i \rangle$ denotes a signed integer whereas $\langle \text{value} \rangle$, $\langle \text{first} \rangle$, $\langle \text{last} \rangle$, $\langle \text{length} \rangle$ and $\langle i \text{ expr} \rangle$ stand for raw integers or more general numerical expressions that are evaluated beforehand.

Resolution occurs only when requested and the result is cached for performance reason.

4.1 Range overlay queries

We give the resolution of queries after named overlay sets are defined.

$\langle \text{ref} \rangle = \langle \text{first} \rangle$: as well as $\langle \text{first} \rangle ::$ define a range limited from below:

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{first} \rangle -$
$\langle \text{ref} \rangle.1$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{first} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{first} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{first} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{first} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{first} \rangle$

Notice that $\langle \text{ref} \rangle.\text{previous}$ and $\langle \text{ref} \rangle.0$ are most of the time synonyms.

$\langle \text{ref} \rangle = \langle \text{first} \rangle : \langle \text{last} \rangle$ as well as variants $\langle \text{first} \rangle :: \langle \text{length} \rangle$, $:: \langle \text{length} \rangle : \langle \text{last} \rangle$ or $: \langle \text{last} \rangle :: \langle \text{length} \rangle$, which are equivalent provided $\langle \text{first} \rangle + \langle \text{length} \rangle = \langle \text{last} \rangle + 1$.

Define a range limited from both above and below:

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{first} \rangle - \langle \text{last} \rangle$
$\langle \text{ref} \rangle.1$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{first} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{first} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{first} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{first} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{first} \rangle$
$\langle \text{ref} \rangle.\text{last}$	$\langle \text{last} \rangle$
$\langle \text{ref} \rangle.\text{next}$	$\langle \text{last} \rangle + 1$
$\langle \text{ref} \rangle.\text{length}$	$\langle \text{length} \rangle$

Notice that the resolution of the $\langle \text{ref} \rangle$ overlay query is a beamer range and not an algebraic difference, negative integers do not make sense there while in `beamer` context.

In the frame example below, we use the `\BeanovesResolve` command for the demonstration. It is mainly used for debugging and testing purposes.

```

1 \Beanoves {
2   A = 3:8, % or similarly A = 3::6, A = ::6:8 and A = :8::6
3 }
4 \begin{frame} {Frame \insertframenum} {Slide \insertslidenumber}
5 \ttfamily
6 \BeanovesResolve[show] (A)          == 3-8,
7 \BeanovesResolve[show] (A.1)       == 3,
8 \BeanovesResolve[show] (A.-1)      == 1,
9 \BeanovesResolve[show] (A.previous) == 2,
10 \BeanovesResolve[show] (A.first)   == 3,
11 \BeanovesResolve[show] (A.last)    == 8,
12 \BeanovesResolve[show] (A.next)    == 9,
13 \BeanovesResolve[show] (A.length)  == 6,
14 \end{frame}

```

$\langle \text{ref} \rangle = [\dots]$

$\langle \text{ref} \rangle = \{\dots\}$ See the list range specifiers in section 3.3.3.

4.2 Value counter queries

$\langle \text{ref} \rangle = \langle \text{value} \rangle$ defines a counter value.

overlay query	resolution
$\langle \text{ref} \rangle$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.1$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.2$	$\langle \text{value} \rangle + 1$
$\langle \text{ref} \rangle.\langle i \rangle$	$\langle \text{value} \rangle + \langle i \rangle - 1$
$\langle \text{ref} \rangle[\langle i \text{ expr} \rangle]$	$\langle \text{value} \rangle + \langle i \text{ expr} \rangle - 1$
$\langle \text{ref} \rangle.\text{previous}$	$\langle \text{value} \rangle - 1$
$\langle \text{ref} \rangle.\text{first}$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.\text{last}$	$\langle \text{value} \rangle$
$\langle \text{ref} \rangle.\text{next}$	$\langle \text{value} \rangle + 1$

Additionally, resolution rules are provided for dedicated *overlay queries*. Here, $\langle \text{ref} \rangle$ is considered a standard programming variable:

$\langle \text{ref} \rangle = \langle \text{set expression} \rangle$, resolve $\langle \text{set expression} \rangle$, assign the result to the $\langle \text{ref} \rangle$ and use it. It defines $\langle \text{ref} \rangle$ globally if not already done. Here $\langle \text{set expression} \rangle$ is the longest character sequence with no space³. To remove this limitation, use an embedded query expression or enclose the `clist` item within a pair of braces.

$\langle \text{ref} \rangle += \langle \text{integer expression} \rangle$, resolve $\langle \text{integer expression} \rangle$ into $\langle \text{integer} \rangle$, advance $\langle \text{ref} \rangle$ by $\langle \text{integer} \rangle$ and use the result.

$++\langle \text{ref} \rangle$, increment $\langle \text{ref} \rangle$ by 1 and use it.

$\langle \text{ref} \rangle ++$, use $\langle \text{ref} \rangle$ and then increment it by 1.

This can be used for an indirection.

IN PROGRESS

```

1 \Beanoves {
2   A = 1,
3   B = [ 10 = 100 ],
4   C = 10,
5 }
6 \begin{frame} {Frame \insertframenum} {Slide \insertslidenumber}
7 \ttfamily
8 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.10) == 10,
9 \BeanovesResolve[show] (B.C)      == \BeanovesResolve[show] (B.10) == 100,
10 \BeanovesResolve[show] (A.C+=10) == \BeanovesResolve[show] (A.20) == 20,
11 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.20) == 20,
12 \BeanovesResolve[show] (A.C+=10) == \BeanovesResolve[show] (A.20) == 20+10,
13 \BeanovesResolve[show] (A.C)      == \BeanovesResolve[show] (A.20) == 30,
14 \BeanovesResolve[show] (B.C)      == \BeanovesResolve[show] (B.20) == 20,
15 \end{frame}

```

In order to decrement a counter, one can increment with a negative value, no dedicated syntax is provided yet.

For each new frame, these counters are reset to the value they were initialized with. Sometimes, resetting the counter manually is necessary, for example when managing `tikz` overlay material.

`\BeanovesReset` `\BeanovesReset` [*options*] [$\langle \text{ref}_1 \rangle [= \langle \text{spec}_1 \rangle]$, ..., $\langle \text{ref}_j \rangle [= \langle \text{spec}_j \rangle]$]

This command is very similar to `\Beanoves`, except that a standalone $\langle \text{ref}_i \rangle$ resets the counter to the last value it was initialized with, and that it is meant to be used inside a `frame` environment. Each $= \langle \text{spec}_j \rangle$ is optional and defaults to `=1`. When the `all` option is provided, some internals that were cached for performance reasons are cleared as well.

4.3 The beamer counters

While inside a `frame` environment, it is possible to save the current value of the `beamerpauses` counter that controls whether elements should appear on the current slide. For that, we can execute one of `\Beanoves{\langle \text{ref} \rangle = pauses}` or in a query

³The parser for algebraic expression is very rudimentary.

?(...($\langle\text{ref}\rangle$ =pauses)...). Then later on, we can use ?(...($\langle\text{ref}\rangle$)...) to refer to this saved value in the same frame⁴. Next frame source is an example of usage.

```

1 \begin{frame}
2 \visible<+>{A}\\
3 \visible<+>{B\Beanoves{afterB=pauses}}\\
4 \visible<+>{C}\\
5 \visible<?(afterB)>{other C}\\
6 \visible<?(afterB.previous)>{other B}\\
7 \end{frame}

```

“A” first appears on slide 1, “B” on slide 2 and “C” on slide 3. On line 2, `afterB` takes the value of the `beamerpauses` counter once updated, *id est* 3. “B” and “other B” as well as “C” and “other C” appear at the same time. If the `beamerpauses` counter is not suitable, we can execute instead one of `\Beanoves{ $\langle\text{ref}\rangle$ =slideinframe}` or inside a query ?(...)($\langle\text{ref}\rangle$ =slideinframe)(...). It uses the numerical value of `\insertslideinframe`.

4.4 Multiple queries

It is possible to replace the comma separated list of queries ?($\langle\text{query}_1\rangle$),...,?($\langle\text{query}_j\rangle$) with the shorter single query ?($\langle\text{query}_1\rangle$,..., $\langle\text{query}_j\rangle$).

4.5 Overriding the default resolution

After `\Beanoves{A = 3:8}`, `A.first` is resolved into 3. With a supplemental instruction `\Beanoves{A.first = 10}`, `A.first` is then resolved into 10 whereas `A.1` is still resolved into 3.

4.6 Frame id

Except for very special situations, the *frame ids* can be left unspecified. When no *frame id* was explicitly provided, `beanoves` uses the *last frame id* and, if the resolution fails, an empty *frame id*. At the beginning of each frame, the *last frame id* is set to the *frame id* of the current frame, which is denoted *current frame id* and is empty by default. Then it gets updated after each named reference resolution where a *frame id* is explicitly given. For example, the first time `A.1` reference is resolved within a given frame, it is first translated to $\langle\text{last frame id}\rangle!`A.1`, but when used just after `Y!C.2`, for example, it becomes a shortcut to `Y!A.1` because the *last frame id* is then `Y`.$

In order to set the *frame id* of the current frame to frame $\langle id\rangle$, use the new `beanoves` id option of the `beamer` frame environment.

`beanoves id` `beanoves id= $\langle\text{frame id}\rangle$ ,`

We can use the same $\langle\text{frame id}\rangle$ for different frames to share named overlay sets. Another possibility offered by the `beanoves` package to share named overlay sets is a fall back mechanism, for example when `X!A` cannot be resolved, `!A` is resolved instead.

⁴See [stackexchange](#) for an alternative that needs at least two passes.

4.7 Resolution command

`\BeanovesResolve` `\BeanovesResolve` [`<setup>`] {`<queries>`}

This function resolves the `<queries>`, which are like the argument of `?(<...>)` instructions: a comma separated list of single `<query>`'s. It is mainly used for debugging purposes. The optional `<setup>` is a key–value list:

`show` the result is left into the input stream

`in:N=<command>` the result is stored into `<command>`.

5 Support

See the [source repository](#). One can report issues there.

6 Implementation

Identify the internal prefix ($\text{\LaTeX}3$ DocStrip convention).

1 `<@@=bnvs>`

6.1 Package declarations

```
2 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
3 \ProvidesExplPackage
4   {beanoves}
5   {2025/06/03}
6   {1.0}
7   {Named overlay specifications for beamer}
```

6.2 Overview

Reserved namespace: identifiers containing the case insensitive string `beanoves` or containing the case insensitive string `bnvs` delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into `beamer` specifications based on this data model.

The development is made easier due to a facility layer over `expl`.

6.3 Facility layer: definitions and naming

In order to make the code shorter and easier to read during development, we add a layer over $\text{\LaTeX}3$. The `c` and `v` argument specifiers take a slightly different meaning when used in a function which name contains `bnvs` or `BNVS`. Where $\text{\LaTeX}3$ would transform `l__bnvs_ref_tl` into `\l__bnvs_ref_tl`, `bnvs` will directly transform `ref` into `\l__bnvs_ref_tl`. The type of the local variable used depends on the context and may be `seq` or `int` for example. There are however a pair of exceptions mentioned below. For a better reading experience, “`ref`” will generally stand for `\l__bnvs_ref_tl`, whereas

“path sequence” will generally stand for `\l__bnvs_path_seq`. Other similar shortcuts are used as well.

Functions with BNVS in their names are management functions. They belong to a deeper layer and do not really contain any logic specific to the `beanoves` package.

```
\BNVS:c      \BNVS:c {<cs core name>}
\BNVS_l:cn   \BNVS_l:cn {<local variable core name>} {<type>}
\BNVS_g:cn   \BNVS_g:cn {<global variable core name>} {<type>}
```

These are naming functions internally used to focus on the discriminating part of variable or function names.

```
8 \cs_new:Npn \BNVS:c      #1 { __bnvs_#1 }
9 \cs_new:Npn \BNVS_l:cn #1 #2 { l__bnvs_#1_#2 }
10 \cs_new:Npn \BNVS_g:cn #1 #2 { g__bnvs_#1_#2 }
```

```
\BNVS_use_raw:N \BNVS_use_raw:N {<cs>}
\BNVS_use_raw:c \BNVS_use_raw:c {<cs name>}
\BNVS_use_raw:Nc \BNVS_use_raw:Nc {<function>} {<cs name>}
\BNVS_use_raw:nc \BNVS_use_raw:nc {<tokens>} {<cs name>}
\BNVS_use:c      \BNVS_use:c {<cs core>}
\BNVS_use:Nc     \BNVS_use:Nc {<function>} {<cs core>}
\BNVS_use:nc     \BNVS_use:nc {<tokens>} {<cs core>}
```

`\BNVS_use_raw:c` is a convenient wrapper over `\use:c`. possibly prepended with some code, for debugging and testing. It needs 3 expansion steps just like `\BNVS_use:c`. The other are used to expand `\use:c` enough before usage by `<function>` or `<tokens>`. The first argument of `<function>` has type N. The next token after `<tokens>` will have type N too. `<cs name>` is a full cs name whereas `<cs core>` will be prepended with the appropriate prefix specific to the `beanoves` package.

```
11 \cs_new:Npn \BNVS_use_raw:N #1 { #1 }
12 \cs_new:Npn \BNVS_use_raw:c #1 {
13   \exp_last_unbraced:No
14   \BNVS_use_raw:N { \cs:w #1 \cs_end: }
15 }

16 \cs_new:Npn \BNVS_use:c #1 {
17   \BNVS_use_raw:c { \BNVS:c { #1 } }
18 }

19 \cs_new:Npn \BNVS_use_raw:NN #1 #2 {
20   #1 #2
21 }

22 \cs_new:Npn \BNVS_use_raw:nN #1 #2 {
23   #1 #2
24 }

25 \cs_new:Npn \BNVS_use_raw:Nc #1 #2 {
26   \exp_args:NNc \BNVS_use_raw:NN #1 { #2 }
27 }
```

```

28 \cs_new:Npn \BNVS_use_raw:nc #1 #2 {
29   \exp_last_unbraced:Nno
30   \BNVS_use_raw:nN { #1 } { \cs:w #2 \cs_end: }
31 }

32 \cs_new:Npn \BNVS_use:Nc #1 #2 {
33   \BNVS_use_raw:Nc #1 { \BNVS:c { #2 } }
34 }

35 \cs_new:Npn \BNVS_use:nc #1 #2 {
36   \BNVS_use_raw:nc { #1 } { \BNVS:c { #2 } }
37 }

38 \tl_new:N \l__bnvs_last_unbraced_tl
39 \cs_new:Npn \BNVS_tl_last_unbraced:nv #1 {
40   \tl_set:Nn \l__bnvs_last_unbraced_tl { #1 }
41   \BNVS_tl_use:nc { \exp_last_unbraced:NV \l__bnvs_last_unbraced_tl }
42 }
43 \cs_new:Npn \BNVS_tl_use:nvv #1 #2 {
44   \BNVS_tl_use:nv { \BNVS_tl_use:nv { #1 } { #2 } }
45 }
46 \cs_new:Npn \BNVS_tl_use:nvvv #1 #2 {
47   \BNVS_tl_use:nvv { \BNVS_tl_use:nv { #1 } { #2 } }
48 }

49 \cs_new:Npn \BNVS_log:n #1 { }
50 \cs_generate_variant:Nn \BNVS_log:n { x }

```

6.3.1 Debug

<code>\BNVS_DEBUG_ON:</code>	<code>\BNVS_DEBUG_ON:</code>
<code>\BNVS_DEBUG_OFF:</code>	<code>\BNVS_DEBUG_OFF:</code>
<code>\BNVS_DEBUG_push:nn</code>	<code>\BNVS_DEBUG_push:nn {⟨types on⟩} {⟨types off⟩}</code>
<code>\BNVS_DEBUG_pop:</code>	<code>\BNVS_DEBUG_pop:</code>

These functions activate or deactivate the debug mode. They are only active in the `beanoves-debug` package. Manage debug messaging for one given *⟨type⟩* or *⟨types⟩*. The implementation is not publicly exposed. The *⟨type⟩* is a single letter or `**`.

6.3.2 Facility layer: Functions

<code>\BNVS_new:cpn</code>	<code>\BNVS_new:cpn</code> is like <code>\cs_new:cpn</code> except that the name argument is tagged for <code>beanoves</code>
<code>\BNVS_set:cpn</code>	package. Similarly for <code>\BNVS_set:cpn</code> .

```

51 \cs_new:Npn \BNVS_new:cpn #1 {
52   \cs_new:cpn { \BNVS:c { #1 } }
53 }

54 \cs_new:Npn \BNVS_set:cpn #1 {
55   \cs_set:cpn { \BNVS:c { #1 } }
56 }

```

```

57 \cs_generate_variant:Nn \cs_generate_variant:Nn { c }
58 \cs_new:Npn \BNVS_generate_variant:cn #1 {
59   \cs_generate_variant:cn { \BNVS:c { #1 } }
60 }

```

6.3.3 logging

<pre> \BNVS_warning:n \BNVS_warning:x \BNVS_error:n \BNVS_error:x \BNVS_fatal:n \BNVS_fatal:x </pre>	<pre> \BNVS_warning:n {<message>} \BNVS_error:n {<message>} \BNVS_fatal:n {<message>} </pre> Very rudimentary. No long message available for error recovery.
--	---

```

61 \msg_new:nnn { beanoves } { :n } { #1 }
62 \msg_new:nnn { beanoves } { :nn } { #1~(#2) }

63 \cs_new:Npn \BNVS_warning:n {
64   \msg_warning:nnn { beanoves } { :n }
65 }
66 \cs_new:Npn \BNVS_warning:x {
67   \msg_warning:nnx { beanoves } { :n }
68 }

69 \cs_new:Npn \BNVS_error:n {
70   \msg_error:nnn { beanoves } { :n }
71 }
72 \cs_generate_variant:Nn \BNVS_error:n { x }

73 \cs_new:Npn \BNVS_fatal:n {
74   \msg_fatal:nnn { beanoves } { :n }
75 }
76 \cs_new:Npn \BNVS_fatal:x {
77   \msg_fatal:nnx { beanoves } { :n }
78 }
79 \cs_new:Npn \BNVS_fatal_unreachable: {
80   \BNVS_fatal:n { Unreachable }
81 }
82 \cs_new:Npn \BNVS_fatal_unreachable: { }

```

6.3.4 Facility layer: Variables

<pre> \BNVS_N_new:c \BNVS_v_new:c </pre>	<pre> \BNVS_N_new:c {<type>} \BNVS_v_new:c {<type>} </pre>
--	--

Creates a collection of typed utility functions, see usage below. These functions are undefined when no longer used. *<type>* is one of *tl*, *seq*...

```

83 \cs_new:Npn \BNVS_N_new:c #1 {

```

```

84 \cs_new:cpn { BNVS_#1:c } ##1 {
85   1 \BNVS:c{ ##1 } \tl_if_empty:nF { ##1 } { _ } #1
86 }
87 \cs_new:cpn { BNVS_#1_new:c } ##1 {
88   \use:c { #1_new:c } { \use:c { BNVS_#1:c } { ##1 } }
89 }
90 \cs_new:cpn { BNVS_#1_use:c } ##1 {
91   \use:c { \cs:w BNVS_#1:c \cs_end: { ##1 } }
92 }
93 \cs_new:cpn { BNVS_#1_use:Nc } ##1 ##2 {
94   \BNVS_use_raw:Nc
95     ##1 { \cs:w BNVS_#1:c \cs_end: { ##2 } }
96 }
97 \cs_new:cpn { BNVS_#1_use:nc } ##1 ##2 {
98   \BNVS_use_raw:nc
99     { ##1 } { \cs:w BNVS_#1:c \cs_end: { ##2 } }
100 }
101 }

102 \cs_new:Npn \BNVS_v_new:c #1 {
103   \cs_new:cpn { BNVS_#1_use:Nv } ##1 ##2 {
104     \BNVS_use_raw:nc
105       { \exp_args:Nv ##1 }
106       { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
107   }
108   \cs_new:cpn { BNVS_#1_use:cv } ##1 ##2 {
109     \BNVS_use_raw:nc
110       { \exp_args:NnV \BNVS_use:c { ##1 } }
111       { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
112   }
113   \cs_new:cpn { BNVS_#1_use:nv } ##1 ##2 {
114     \BNVS_use_raw:nc
115       { \exp_args:NnV \use:n { ##1 } }
116       { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
117   }
118 }

119 \BNVS_N_new:c { bool }
120 \BNVS_N_new:c { int }
121 \BNVS_v_new:c { int }
122 \BNVS_N_new:c { tl }
123 \BNVS_v_new:c { tl }
124 \cs_new:Npn \BNVS_tl_use:Nvv #1 #2 {
125   \BNVS_tl_use:nv { \BNVS_tl_use:Nv #1 { #2 } }
126 }

127 \cs_new:Npn \BNVS_tl_use:Nvvv #1 #2 {
128   \BNVS_tl_use:nvv { \BNVS_tl_use:Nv #1 { #2 } }
129 }

130 \cs_new:Npn \BNVS_tl_use:Nvvvv #1 #2 {
131   \BNVS_tl_use:nvvv { \BNVS_tl_use:Nv #1 { #2 } }
132 }

133 \BNVS_N_new:c { str }
134 \BNVS_v_new:c { str }

```

```

135 \BNVS_N_new:c { seq }
136 \BNVS_v_new:c { seq }
137 \cs_undefine:N \BNVS_N_new:c

```

\BNVS_use:Ncn \BNVS_use:Ncn *<function>* *{<core name>}* *{<type>}*

```

138 \cs_new:Npn \BNVS_use:Ncn #1 #2 #3 {
139   \BNVS_use_raw:c { BNVS_#3_use:Nc }   #1   { #2 }
140 }

141 \cs_new:Npn \BNVS_use:ncn #1 #2 #3 {
142   \BNVS_use_raw:c { BNVS_#3_use:nc } { #1 } { #2 }
143 }

144 \cs_new:Npn \BNVS_use:Nvn #1 #2 #3 {
145   \BNVS_use_raw:c { BNVS_#3_use:Nv }   #1   { #2 }
146 }

147 \cs_new:Npn \BNVS_use:nvn #1 #2 #3 {
148   \BNVS_use_raw:c { BNVS_#3_use:nv } { #1 } { #2 }
149 }

150 \cs_new:Npn \BNVS_use:Ncncn #1 #2 #3 {
151   \BNVS_use:ncn {
152     \BNVS_use:Ncn   #1   { #2 } { #3 }
153   }
154 }

155 \cs_new:Npn \BNVS_use:ncncn #1 #2 #3 {
156   \BNVS_use:ncn {
157     \BNVS_use:ncn { #1 } { #2 } { #3 }
158   }
159 }

160 \cs_new:Npn \BNVS_use:Nvncn #1 #2 #3 {
161   \BNVS_use:ncn {
162     \BNVS_use:Nvn   #1   { #2 } { #3 }
163   }
164 }

165 \cs_new:Npn \BNVS_use:nvncn #1 #2 #3 {
166   \BNVS_use:ncn {
167     \BNVS_use:nvn { #1 } { #2 } { #3 }
168   }
169 }

170 \cs_new:Npn \BNVS_use:Ncncncn #1 #2 #3 #4 #5 {
171   \BNVS_use:ncn {
172     \BNVS_use:Ncncn   #1   { #2 } { #3 } { #4 } { #5 }
173   }
174 }

175 \cs_new:Npn \BNVS_use:ncncncn #1 #2 #3 #4 #5 {
176   \BNVS_use:ncn {
177     \BNVS_use:ncncn { #1 } { #2 } { #3 } { #4 } { #5 }
178   }
179 }

```

`\BNVS_new_c:cn \BNVS_new_c:nc {<type>} {<core name>}`

```
180 \cs_new:Npn \BNVS_new_c:nc #1 #2 {
181   \BNVS_new_cpn { #1_#2:c } {
182     \BNVS_use_raw:c { BNVS_#1_use:nc } { \BNVS_use_raw:c { #1_#2:N } }
183   }
184 }

185 \cs_new:Npn \BNVS_new_cn:nc #1 #2 {
186   \BNVS_new_cpn { #1_#2:cn } ##1 {
187     \BNVS_use:ncn { \BNVS_use_raw:c { #1_#2:Nn } } { ##1 } { #1 }
188   }
189 }

190 \cs_new:Npn \BNVS_new_cnn:ncN #1 #2 #3 {
191   \BNVS_new_cpn { #2:cnn } ##1 {
192     \BNVS_use:Ncn { #3 } { ##1 } { #1 }
193   }
194 }

195 \cs_new:Npn \BNVS_new_cnn:nc #1 #2 {
196   \BNVS_use_raw:nc {
197     \BNVS_new_cnn:ncN { #1 } { #1_#2 }
198   } { #1_#2:Nnn }
199 }

200 \cs_new:Npn \BNVS_new_cnv:ncN #1 #2 #3 {
201   \BNVS_new_cpn { #2:cnv } ##1 ##2 {
202     \BNVS_tl_use:nv {
203       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
204     }
205   }
206 }

207 \cs_new:Npn \BNVS_new_cnv:nc #1 #2 {
208   \BNVS_use_raw:nc {
209     \BNVS_new_cnv:ncN { #1 } { #1_#2 }
210   } { #1_#2:Nnn }
211 }

212 \cs_new:Npn \BNVS_new_cnx:ncN #1 #2 #3 {
213   \BNVS_new_cpn { #2:cnx } ##1 ##2 {
214     \exp_args:Nnx \use:n {
215       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
216     }
217   }
218 }

219 \cs_new:Npn \BNVS_new_cnx:nc #1 #2 {
220   \BNVS_use_raw:nc {
221     \BNVS_new_cnx:ncN { #1 } { #1_#2 }
222   } { #1_#2:Nnn }
223 }
```

```

224 \cs_new:Npn \BNVS_new_cc:ncNn #1 #2 #3 #4 {
225   \BNVS_new:cpn { #2:cc } ##1 ##2 {
226     \BNVS_use:Ncncn #3 { ##1 } { #1 } { ##2 } { #4 }
227   }
228 }

229 \cs_new:Npn \BNVS_new_cc:ncn #1 #2 {
230   \BNVS_use_raw:nc {
231     \BNVS_new_cc:ncNn { #1 } { #1_#2 }
232   } { #1_#2:NN }
233 }

234 \cs_new:Npn \BNVS_new_cc:nc #1 #2 {
235   \BNVS_new_cc:ncn { #1 } { #2 } { #1 }
236 }

237 \cs_new:Npn \BNVS_new_cn:ncNn #1 #2 #3 #4 {
238   \BNVS_new:cpn { #2:cn } ##1 {
239     \BNVS_use:Ncn #3 { ##1 } { #1 }
240   }
241 }

242 \cs_new:Npn \BNVS_new_cn:ncn #1 #2 {
243   \BNVS_use_raw:nc {
244     \BNVS_new_cn:ncNn { #1 } { #1_#2 }
245   } { #1_#2:Nn }
246 }

247 \cs_new:Npn \BNVS_new_cv:ncNn #1 #2 #3 #4 {
248   \BNVS_new:cpn { #2:cv } ##1 ##2 {
249     \BNVS_use:nvn {
250       \BNVS_use:Ncn #3 { ##1 } { #1 }
251     } { ##2 } { #4 }
252   }
253 }

254 \cs_new:Npn \BNVS_new_cv:ncn #1 #2 {
255   \BNVS_use_raw:nc {
256     \BNVS_new_cv:ncNn { #1 } { #1_#2 }
257   } { #1_#2:Nn }
258 }

259 \cs_new:Npn \BNVS_new_cv:nc #1 #2 {
260   \BNVS_new_cv:ncn { #1 } { #2 } { #1 }
261 }

262 \cs_new:Npn \BNVS_l_use:Ncn #1 #2 #3 {
263   \BNVS_use_raw:Nc #1 { \BNVS_l:cn { #2 } { #3 } }
264 }

265 \cs_new:Npn \BNVS_l_use:ncn #1 #2 #3 {
266   \BNVS_use_raw:nc { #1 } { \BNVS_l:cn { #2 } { #3 } }
267 }

```



```

268 \cs_new:Npn \BNVS_g_use:Ncn #1 #2 #3 {
269   \BNVS_use_raw:Nc    #1    { \BNVS_g:cn { #2 } { #3 } }
270 }

271 \cs_new:Npn \BNVS_g_use:ncn #1 #2 #3 {
272   \BNVS_use_raw:nc { #1 } { \BNVS_g:cn { #2 } { #3 } }
273 }

```

```

\BNVS_new_conditional:cpnn \BNVS_new_conditional:cpnn {<core>} <parameter> {<conditions>} {<code>}

```

```

274 \cs_generate_variant:Nn \prg_new_conditional:Npnn { c }

275 \cs_new:Npn \BNVS_new_conditional:cpnn #1 {
276   \prg_new_conditional:cpnn { \BNVS:c { #1 } }
277 }
278 \cs_generate_variant:Nn \prg_generate_conditional_variant:Nnn { c }
279 \cs_new:Npn \BNVS_generate_conditional_variant:cnn #1 {
280   \prg_generate_conditional_variant:cnn { \BNVS:c { #1 } }
281 }

282 \cs_new:Npn \BNVS_new_conditional_vn:cNnn #1 #2 #3 #4 {
283   \BNVS_new_conditional:cpnn { #1:vn } ##1 ##2 { #4 } {
284     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
285       \prg_return_true:
286     } {
287       \prg_return_false:
288     }
289   }
290 }

291 \cs_new:Npn \BNVS_new_conditional_vn:cnn #1 #2 {
292   \BNVS_use:nc {
293     \BNVS_new_conditional_vn:cNnn { #1 }
294   } { #1:nn TF } { #2 }
295 }

296 \cs_new:Npn \BNVS_new_conditional_vc:cNnn #1 #2 #3 #4 {
297   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #4 } {
298     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
299       \prg_return_true:
300     } {
301       \prg_return_false:
302     }
303   }
304 }

305 \cs_new:Npn \BNVS_new_conditional_vc:cnn #1 {
306   \BNVS_use:nc {
307     \BNVS_new_conditional_vc:cNnn { #1 }
308   } { #1:ncTF }
309 }

```

```

310 \cs_new:Npn \BNVS_new_conditional_vvc:cNnnn #1 #2 #3 #4 #5 {
311   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #5 } {
312     \BNVS_use:nvn {
313       \BNVS_use:Nvn #2 { ##1 } { #3 }
314     } { ##2 } { #4 } { ##3 } {
315       \prg_return_true:
316     } {
317       \prg_return_false:
318     }
319   }
320 }

321 \cs_new:Npn \BNVS_new_conditional_vvc:cnnn #1 {
322   \BNVS_use:nc {
323     \BNVS_new_conditional_vvc:cNnnn { #1 }
324   } { #1:nncTF }
325 }

326 \cs_new:Npn \BNVS_new_conditional_vc:cNn #1 #2 #3 {
327   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #3 } {
328     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } {
329       \prg_return_true:
330     } {
331       \prg_return_false:
332     }
333   }
334 }

335 \cs_new:Npn \BNVS_new_conditional_vc:cn #1 {
336   \BNVS_use:nc {
337     \BNVS_new_conditional_vc:cNn { #1 }
338   } { #1:ncTF }
339 }

340 \cs_new:Npn \BNVS_new_conditional_vvc:cNn #1 #2 #3 {
341   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #3 } {
342     \BNVS_tl_use:nv {
343       \BNVS_tl_use:Nv #2 { ##1 }
344     } { ##2 } { ##3 } {
345       \prg_return_true:
346     } {
347       \prg_return_false:
348     }
349   }
350 }

351 \cs_new:Npn \BNVS_new_conditional_vvc:cn #1 {
352   \BNVS_use:nc {
353     \BNVS_new_conditional_vvc:cNn { #1 }
354   } { #1:nncTF }
355 }

```

6.3.5 Regex

```

356 \cs_new:Npn \BNVS_regex_use:Nc #1 #2 {
357   \BNVS_use_raw:Nc #1 { c \BNVS:c { #2 } _regex }
358 }

```

6.3.6 Token lists

<u>__bnvs_tl_clear:c</u>	<u>__bnvs_tl_clear:c {<core key tl>}</u>
<u>__bnvs_tl_use:c</u>	<u>__bnvs_tl_use:c {<core>}</u>
<u>__bnvs_tl_set_eq:cc</u>	<u>__bnvs_tl_count:c {<core>}</u>
<u>__bnvs_tl_set:cn</u>	<u>__bnvs_tl_set_eq:cc {<lhs core name>} {<rhs core name>}</u>
<u>__bnvs_tl_set:(cv cx ce)</u>	<u>__bnvs_tl_set:cn {<core>} {<tl>}</u>
<u>__bnvs_tl_put_left:cn</u>	<u>__bnvs_tl_set:cv {<core>} {<value core name>}</u>
<u>__bnvs_tl_put_right:cn</u>	<u>__bnvs_tl_put_left:cn {<core>} {<tl>}</u>
<u>__bnvs_tl_put_right:(cx ce cv)</u>	<u>__bnvs_tl_put_right:cn {<core>} {<tl>}</u>
	<u>__bnvs_tl_put_right:cv {<core>} {<value core name>}</u>

These are shortcuts to

- \tl_clear:c {l__bnvs_<core>_tl}
- \tl_use:c {l__bnvs_<core>_tl}
- \tl_set_eq:cc {l__bnvs_<lhs core>_tl}{l__bnvs_<rhs core>_tl}
- \tl_set:cv {l__bnvs_<core>_tl}{l__bnvs_<value core>_tl}
- \tl_set:cx {l__bnvs_<core>_tl}{<tl>}
- \tl_put_left:cn {l__bnvs_<core>_tl}{<tl>}
- \tl_put_right:cn {l__bnvs_<core>_tl}{<tl>}
- \tl_put_right:cv {l__bnvs_<core>_tl}{l__bnvs_<value core>_tl}

<u>\BNVS_new_conditional_vnc:cn</u>	<u>\BNVS_new_conditional_vnc:cn {<core>} {<conditions>}</u>
-------------------------------------	---

Creates <core>:vncTF from <core>:nncTF.

```

359 \cs_new:Npn \BNVS_new_conditional_vnc:cNn #1 #2 #3 {
360   \BNVS_new_conditional:cpnn { #1:vnc } ##1 ##2 ##3 { #3 } {
361     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } { ##3 } {
362       \prg_return_true:
363     } {
364       \prg_return_false:
365     }
366   }
367 }

```

```

368 \cs_new:Npn \BNVS_new_conditional_vnc:cn #1 {
369   \BNVS_use:nc {
370     \BNVS_new_conditional_vnc:cNn { #1 }
371   } { #1:nnctf }
372 }

```

\BNVS_new_conditional_vvnc:cn \BNVS_new_conditional_vvnc:cn {<core>} {<conditions>}

Creates <core>:vvncTF from <core>:nnctf.

```

373 \cs_new:Npn \BNVS_new_conditional_vvnc:cNn #1 #2 #3 {
374   \BNVS_new_conditional:cpnn { #1:vvnc } ##1 ##2 ##3 ##4 { #3 } {
375     \BNVS_tl_use:nv {
376       \BNVS_tl_use:Nv #2 { ##1 }
377     } { ##2 } { ##3 } { ##4 } {
378       \prg_return_true:
379     } {
380       \prg_return_false:
381     }
382   }
383 }

384 \cs_new:Npn \BNVS_new_conditional_vvnc:cn #1 {
385   \BNVS_use:nc {
386     \BNVS_new_conditional_vvnc:cNn { #1 }
387   } { #1:nnctf }
388 }

389 \cs_new:Npn \BNVS_new_conditional_vvvc:cNn #1 #2 #3 {
390   \BNVS_new_conditional:cpnn { #1:vvvc } ##1 ##2 ##3 ##4 { #3 } {
391     \BNVS_tl_use:nvv {
392       \BNVS_tl_use:Nv #2 { ##1 }
393     } { ##2 } { ##3 } { ##4 } {
394       \prg_return_true:
395     } {
396       \prg_return_false:
397     }
398   }
399 }

```

\BNVS_new_conditional_vvvc:cn \BNVS_new_conditional_vvvc:cn {<core>} {<conditions>}

Creates <core>:vvvcTF from <core>:nnctf.

```

400 \cs_new:Npn \BNVS_new_conditional_vvvc:cn #1 {
401   \BNVS_use:nc {
402     \BNVS_new_conditional_vvvc:cNn { #1 }
403   } { #1:nnctf }
404 }

```

```

405 \cs_new:Npn \BNVS_new_tl_c:c {
406   \BNVS_new_c:nc { tl }
407 }
408 \BNVS_new_tl_c:c { clear }
409 \BNVS_new_tl_c:c { use }
410 \BNVS_new_tl_c:c { count }

411 \BNVS_new:cpn { tl_set_eq:cc } #1 #2 {
412   \BNVS_use:ncncn { \tl_set_eq:NN } { #1 } { tl } { #2 } { tl }
413 }

414 \cs_new:Npn \BNVS_new_tl_cn:c {
415   \BNVS_new_cn:nc { tl }
416 }

417 \cs_new:Npn \BNVS_new_tl_cv:c #1 {
418   \BNVS_new_cv:ncn { tl } { #1 } { tl }
419 }
420 \BNVS_new_tl_cn:c { set }
421 \BNVS_new_tl_cv:c { set }

422 \BNVS_new:cpn { tl_set:cx } {
423   \exp_args:Nnx \__bnvs_tl_set:cn
424 }

425 \BNVS_new:cpn { tl_set:ce } {
426   \exp_args:Nne \__bnvs_tl_set:cn
427 }
428 \BNVS_new_tl_cn:c { put_right }
429 \BNVS_new_tl_cv:c { put_right }
430 % \BNVS_generate_variant:cn { tl_put_right:cn } { cx }

431 \BNVS_new:cpn { tl_put_right:cx } {
432   \exp_args:Nnnx \BNVS_use:c { tl_put_right:cn }
433 }

434 \BNVS_new:cpn { tl_put_right:ce } {
435   \exp_args:Nnne \BNVS_use:c { tl_put_right:cn }
436 }
437 \BNVS_new_tl_cn:c { put_left }
438 \BNVS_new_tl_cv:c { put_left }
439 % \BNVS_generate_variant:cn { tl_put_left:cn } { cx }

440 \BNVS_new:cpn { tl_put_left:cx } {
441   \exp_args:Nnnx \BNVS_use:c { tl_put_left:cn }
442 }

```

<u>__bnvs_tl_if_empty:cTF</u>	<u>__bnvs_tl_if_empty:cTF</u>	<code>{\<core>}</code>	<code>{\<yes:>}</code>	<code>{\<no:>}</code>
<u>__bnvs_tl_if_blank:vTF</u>	<u>__bnvs_tl_if_blank:vTF</u>	<code>{\<core>}</code>	<code>{\<yes:>}</code>	<code>{\<no:>}</code>
<u>__bnvs_tl_if_eq:cnTF</u>	<u>__bnvs_tl_if_eq:cnTF</u>	<code>{\<core>}</code>	<code>{\<tl>}</code>	<code>{\<yes:>}</code> <code>{\<no:>}</code>

These are shortcuts to

- `\tl_if_empty:cTF {l__bnvs_<core>_tl} {\<yes:>} {\<no:>}`
- `\tl_if_eq:cnTF {l__bnvs_<core>_tl}{\<tl>} {\<yes:>} {\<no:>}`

```

443 \cs_new:Npn \BNVS_new_conditional_c:ncNn #1 #2 #3 #4 {
444   \BNVS_new_conditional:cpnn { #2 } ##1 { #4 } {
445     \BNVS_use:Ncn #3 { ##1 } { #1 } {
446       \prg_return_true:
447     } {
448       \prg_return_false:
449     }
450   }
451 }

452 \cs_new:Npn \BNVS_new_conditional_c:ncn #1 #2 {
453   \BNVS_use_raw:nc {
454     \BNVS_new_conditional_c:ncNn { #1 } { #1_#2:c }
455   } { #1_#2:NTF }
456 }
457 \BNVS_new_conditional_c:ncn { t1 } { if_empty } { p, T, F, TF }

458 \BNVS_new_conditional:cpnn { t1_if_blank:v } #1 { T, F, TF } {
459   \BNVS_t1_use:Nv \t1_if_blank:nTF { #1 } {
460     \prg_return_true:
461   } {
462     \prg_return_false:
463   }
464 }

465 \cs_new:Npn \BNVS_new_conditional_cn:ncNn #1 #2 #3 #4 {
466   \BNVS_new_conditional:cpnn { #2:cn } ##1 ##2 { #4 } {
467     \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } {
468       \prg_return_true:
469     } {
470       \prg_return_false:
471     }
472   }
473 }

474 \cs_new:Npn \BNVS_new_conditional_cn:ncn #1 #2 {
475   \BNVS_use_raw:nc {
476     \BNVS_new_conditional_cn:ncNn { #1 } { #1_#2 }
477   } { #1_#2:NnTF }
478 }
479 \BNVS_new_conditional_cn:ncn { t1 } { if_eq } { T, F, TF }

480 \cs_new:Npn \BNVS_new_conditional_cv:ncNn #1 #2 #3 #4 {
481   \BNVS_new_conditional:cpnn { #2:cv } ##1 ##2 { #4 } {
482     \BNVS_use:nvn {
483       \BNVS_use:Ncn #3 { ##1 } { #1 }
484     } { ##2 } { #1 } {
485       \prg_return_true:
486     } {
487       \prg_return_false:
488     }
489   }
490 }

```

```

491 \cs_new:Npn \BNVS_new_conditional_cv:ncn #1 #2 {
492   \BNVS_use_raw:nc {
493     \BNVS_new_conditional_cv:ncNn { #1 } { #1_#2 }
494   } { #1_#2:NnTF }
495 }
496 \BNVS_new_conditional_cv:ncn { t1 } { if_eq } { T, F, TF }

497 \BNVS_new:cpn { gput_right:cccn } #1 #2 #3 {
498   \tl_gput_right:cn { \_bnvs_g_IPK:w { #1 } { #2 } { #3 } }
499 }

```

6.3.7 Strings

_bnvs_str_if_eq:vvTF _bnvs_str_if_eq:vvTF {<core>} {<tl>} {<yes:>} {<no:>}

These are shortcuts to

- \str_if_eq:ccTF {l_bnvs_<core>_tl}{<yes:>} {<no:>}

```

500 \cs_new:Npn \BNVS_new_conditional_vv:cNn #1 #2 #3 {
501   \BNVS_new_conditional:cpnn { #1:vv } ##1 ##2 { #3 } {
502     \BNVS_tl_use:nv {
503       \BNVS_tl_use:Nv #2 { ##1 }
504     } { ##2 } {
505       \prg_return_true:
506     } {
507       \prg_return_false:
508     }
509   }
510 }

511 \cs_new:Npn \BNVS_new_conditional_vv:cn #1 {
512   \BNVS_use:nc {
513     \BNVS_new_conditional_vvnc:cNn { #1 }
514   } { #1:nnTF }
515 }

516 \cs_new:Npn \BNVS_new_conditional_vn:ncNn #1 #2 #3 #4 {
517   \BNVS_new_conditional:cpnn { #2:vn } ##1 ##2 { #4 } {
518     \BNVS_use:Nvn #3 { ##1 } { #1 } { ##2 } {
519       \prg_return_true:
520     } {
521       \prg_return_false:
522     }
523   }
524 }

525 \cs_new:Npn \BNVS_new_conditional_vn:ncn #1 #2 {
526   \BNVS_use_raw:nc {
527     \BNVS_new_conditional_vn:ncNn { #1 } { #1_#2 }
528   } { #1_#2:nnTF }
529 }
530 \BNVS_new_conditional_vn:ncn { str } { if_eq } { T, F, TF }

```

```

531 \cs_new:Npn \BNVS_new_conditional_vv:ncNn #1 #2 #3 #4 {
532   \BNVS_new_conditional:cpnn { #2:vv } ##1 ##2 { #4 } {
533     \BNVS_use:nvn {
534       \BNVS_use:Nvn #3 { ##1 } { #1 }
535     } { ##2 } { #1 } {
536       \prg_return_true:
537     } {
538       \prg_return_false:
539     }
540   }
541 }

542 \cs_new:Npn \BNVS_new_conditional_vv:ncn #1 #2 {
543   \BNVS_use_raw:nc {
544     \BNVS_new_conditional_vv:ncNn { #1 } { #1_#2 }
545   } { #1_#2:nnTF }
546 }
547 \BNVS_new_conditional_vv:ncn { str } { if_eq } { T, F, TF }

```

6.3.8 Sequences

__bnvs_seq_count:c	__bnvs_seq_new:c {<core>}
__bnvs_seq_clear:c	__bnvs_seq_count:c {<core>}
__bnvs_seq_set_eq:cc	__bnvs_seq_clear:c {<core>}
__bnvs_seq_gset_eq:cc	__bnvs_seq_set_eq:cc {<core ₁ >} {<core ₂ >}
__bnvs_seq_use:cn	__bnvs_seq_use:cn {<core>} {<separator>}
__bnvs_seq_item:cn	__bnvs_seq_item:cn {<core>} {<integer expression>}
__bnvs_seq_remove_all:cn	__bnvs_seq_remove_all:cn {<core>} {<tl>}
__bnvs_seq_put_left:cv	__bnvs_seq_put_right:cn {<seq core>} {<tl>}
__bnvs_seq_put_right:cn	__bnvs_seq_put_right:cv {<seq core>} {<tl core>}
__bnvs_seq_put_right:cv	__bnvs_seq_set_split:cnn {<seq core>} {<tl>} {<separator>}
__bnvs_seq_set_split:cnn	__bnvs_seq_pop_left:cc {<core ₁ >} {<core ₂ >}
__bnvs_seq_set_split:(cnv cnx)	
__bnvs_seq_pop_left:cc	

These are shortcuts to

- \seq_set_eq:cc {l__bnvs_<core₁>_seq} {l__bnvs_<core₂>_seq}
- \seq_count:c {l__bnvs_<core>_seq}
- \seq_use:cn {l__bnvs_<core>_seq} {<separator>}
- \seq_item:cn {l__bnvs_<core>_seq} {<integer expression>}
- \seq_remove_all:cn {l__bnvs_<core>_seq} {<tl>}
- \seq_clear:c {l__bnvs_<core>_seq}
- \seq_put_right:cv {l__bnvs_<core>_seq} {l__bnvs_<core>_tl}
- \seq_set_split:cnn {l__bnvs_<core>_seq} {l__bnvs_<core>_tl}\KWNmeta{tl}


```

548 \BNVS_new_c:nc { seq } { count }
549 \BNVS_new_c:nc { seq } { clear }
550 \BNVS_new_cn:nc { seq } { use }
551 \BNVS_new_cn:nc { seq } { item }
552 \BNVS_new_cn:nc { seq } { remove_all }
553 \BNVS_new_cn:nc { seq } { map_inline }
554 \BNVS_new_cc:nc { seq } { set_eq }
555 \BNVS_new_cc:nc { seq } { gset_eq }
556 \BNVS_new_cv:ncn { seq } { put_left } { t1 }
557 \BNVS_new_cn:ncn { seq } { put_right } { t1 }
558 \BNVS_new_cv:ncn { seq } { put_right } { t1 }
559 \BNVS_new_cnn:nc { seq } { set_split }
560 \BNVS_new_cnv:nc { seq } { set_split }
561 \BNVS_new_cnx:nc { seq } { set_split }
562 \BNVS_new_cc:ncn { seq } { pop_left } { t1 }
563 \BNVS_new_cc:ncn { seq } { pop_right } { t1 }

```

```

\__bnvs_seq_if_empty:cTF \__bnvs_seq_if_empty:cTF {<seq core>} {<yes:>} {<no:>}
\__bnvs_seq_get_right:ccTF \__bnvs_seq_get_right:ccTF {<seq core>} {<t1 core>} {<yes:>} {<no:>}
\__bnvs_seq_pop_left:ccTF
\__bnvs_seq_pop_right:ccTF

```

```

564 \cs_new:Npn \BNVS_new_conditional_cc:ncnn #1 #2 #3 #4 {
565   \BNVS_new_conditional:cpnn { #1_#2:cc } ##1 ##2 { #4 } {
566     \BNVS_use:ncncn {
567       \BNVS_use_raw:c { #1_#2:NNTF }
568     } { ##1 } { #1 } { ##2 } { #3 } {
569       \prg_return_true:
570     } {
571       \prg_return_false:
572     }
573   }
574 }
575 \BNVS_new_conditional_c:ncn { seq } { if_empty } { T, F, TF }
576 \BNVS_new_conditional_cc:ncnn
577 { seq } { get_right } { t1 } { T, F, TF }
578 \BNVS_new_conditional_cc:ncnn
579 { seq } { pop_left } { t1 } { T, F, TF }
580 \BNVS_new_conditional_cc:ncnn
581 { seq } { pop_right } { t1 } { T, F, TF }

```

6.3.9 Integers

<code>__bnvs_int_new:c</code>	<code>__bnvs_int_new:c</code>	<code>{\core}</code>
<code>__bnvs_int_use:c</code>	<code>__bnvs_int_use:c</code>	<code>{\core}</code>
<code>__bnvs_int_zero:c</code>	<code>__bnvs_int_incr:c</code>	<code>{\core}</code>
<code>__bnvs_int_inc:c</code>	<code>__bnvs_int_decr:c</code>	<code>{\core}</code>
<code>__bnvs_int_decr:c</code>	<code>__bnvs_int_set:cn</code>	<code>{\core}</code> <code>{\value}</code>
<code>__bnvs_int_set:cn</code>	These are shortcuts to	
<code>__bnvs_int_set:cv</code>		

- `\int_new:c` `{l__bnvs_<core>_int}`
- `\int_use:c` `{l__bnvs_<core>_int}`
- `\int_incr:c` `{l__bnvs_<core>_int}`
- `\int_decr:c` `{l__bnvs_<core>_int}`
- `\int_set:cn` `{l__bnvs_<core>_int}` `<value>`

```

582 \BNVS_new_c:nc { int } { new }
583 \BNVS_new_c:nc { int } { use }
584 \BNVS_new_c:nc { int } { zero }
585 \BNVS_new_c:nc { int } { incr }
586 \BNVS_new_c:nc { int } { decr }
587 \BNVS_new_cn:nc { int } { set }
588 \BNVS_new_cv:ncn { int } { set } { int }

```

6.4 Debug facilities

Typesetting file `beanoves.dtx` creates both `beanoves` and `beanoves-debug` style files. The former is intended for everyday use whereas the latter contains supplemental debugging and testing facilities which are intentionally left undocumented. For example, we have aliases for `\group_begin:` and `\group_end:` to allow the display of supplemental informations while debugging. We also have some techniques for coverage as well as inline testing.

6.5 Local variables

We make heavy use of local variables and function scopes. Many functions are executed within a `TEX` group, which ensures no name collision with the caller stack. The number of variables used has not been optimized, nor the `TEX` groups used. Optimization often goes against readability. Next local variables are used by different functions. These are one word letter variables.

`\l__bnvs_I_tl`: A frame identifier

`\l__bnvs_J_tl`: The last frame identifier

`\l__bnvs_K_tl`: ! as regex caught group, if non empty.

`\l__bnvs_S_tl`: A short name, a regex caught group.

`\l__bnvs_P_tl`: A path, a regex caught group.

`\l__bnvs_N_tl`: The representation of an unsigned decimal integer.

```
589 \tl_new:N \l__bnvs_J_tl
590 \tl_new:N \l__bnvs_I_tl
591 \tl_new:N \l__bnvs_K_tl
592 \tl_new:N \l__bnvs_S_tl
593 \tl_new:N \l__bnvs_P_tl
594 \tl_new:N \l__bnvs_R_tl
595 \tl_new:N \l__bnvs_D_tl
596 \tl_new:N \l__bnvs_B_tl
597 \tl_new:N \l__bnvs_N_tl
598 \tl_new:N \l__bnvs_T_tl
599 \tl_new:N \l__bnvs_U_tl
600 \tl_new:N \l__bnvs_V_tl
601 \tl_new:N \l__bnvs_A_tl
602 \tl_new:N \l__bnvs_C_tl
603 \tl_new:N \l__bnvs_F_tl
604 \tl_new:N \l__bnvs_L_tl
605 \tl_new:N \l__bnvs_Z_tl
606 \tl_new:N \l__bnvs_Q_tl
607 \tl_new:N \l__bnvs_a_tl
608 \tl_new:N \l__bnvs_z_tl
609 \tl_new:N \l__bnvs_l_tl
610 \tl_new:N \l__bnvs_r_tl
611 \tl_new:N \l__bnvs_n_tl
612 \tl_new:N \l__bnvs_ref_tl
613 \tl_new:N \l__bnvs_v_tl
614 \tl_new:N \l__bnvs_b_tl
615 \tl_new:N \l__bnvs_c_tl
616 \tl_new:N \l__bnvs_ans_tl
617 \tl_new:N \l__bnvs_base_tl
618 \tl_new:N \l__bnvsroup_tl
619 \tl_new:N \l__bnvs_scan_tl
620 \tl_new:N \l__bnvs_query_tl
621 \tl_new:N \l__bnvs_n_incr_tl
622 \tl_new:N \l__bnvs_incr_tl
623 \tl_new:N \l__bnvs_plus_tl
624 \tl_new:N \l__bnvs_rhs_tl
625 \tl_new:N \l__bnvs_post_tl
626 \tl_new:N \l__bnvs_suffix_tl
627 \tl_new:N \l__bnvs_index_tl
628 \int_new:N \g__bnvs_call_int
629 \int_new:N \l__bnvs_i_int
630 \seq_new:N \g__bnvs_def_seq
631 \seq_new:N \l__bnvs_a_seq
632 \seq_new:N \l__bnvs_b_seq
633 \seq_new:N \l__bnvs_ans_seq
634 \seq_new:N \l__bnvs_Q_seq
635 \seq_new:N \l__bnvs_R_seq
636 \seq_new:N \l__bnvs_match_seq
637 \seq_new:N \l__bnvs_split_seq
638 \seq_new:N \l__bnvs_P_seq
```

```

639 \bool_new:N \l__bnvs_parse_bool
640 \bool_set_false:N \l__bnvs_parse_bool
641 \bool_new:N \l__bnvs_deep_bool
642 \bool_set_false:N \l__bnvs_deep_bool
643 \bool_new:N \l__bnvs_no_range_bool
644 \bool_set_false:N \l__bnvs_no_range_bool

645 \BNVS_new:cpn { tl_clear_ans: } {
646   \__bnvs_tl_clear:c { ans }
647 }

648 \cs_new:Npn \BNVS_error_ans:x {
649   \__bnvs_tl_put_right:cn { ans } 0
650   \BNVS_error:x
651 }

```

In order to implement the provide feature, we add getters and setters

```

652 \BNVS_new:cpn { set_true:c } #1 {
653   \exp_args:Nc \bool_set_true:N { \BNVS_1:cn { #1 } { bool } }
654 }

655 \BNVS_new:cpn { set_false:c } #1 {
656   \exp_args:Nc \bool_set_false:N { \BNVS_1:cn { #1 } { bool } }
657 }

```

6.6 Infinite loop management

Unending recursivity is managed here.

`\g__bnvs_call_int` Some functions calls, as well as some loop bodies, decrement this counter. When this counter reaches 0, an error is raised or a computation is aborted.

(End of definition for `\g__bnvs_call_int`.)

```

658 \int_const:Nn \c__bnvs_max_call_int { 8192 }

```

`\__bnvs_greset_call:` `\__bnvs_greset_call:`

Reset globally the call stack counter to its maximum value.

```

659 \BNVS_new:cpn { greset_call: } {
660   \int_gset:Nn \g__bnvs_call_int { \c__bnvs_max_call_int }
661 }

```

`\__bnvs_if_call:TF` `\__bnvs_call_do:TF` `{⟨yes:⟩} {⟨no:⟩}`

Decrement the `\g__bnvs_call_int` counter globally and execute `⟨yes:⟩` if we have not reached 0, `⟨no:⟩` otherwise.

```

662 \BNVS_new_conditional:cpnn { if_call: } { T, F, TF } {
663   \int_gdecr:N \g__bnvs_call_int
664   \int_compare:nNnTF \g__bnvs_call_int > 0 {
665     \prg_return_true:
666   } {
667     \prg_return_false:
668   }
669 }

```

6.7 Overlay specification

6.7.1 Registration

We keep track of the $\langle id \rangle! \langle path \rangle$ combinations and provide looping mechanisms. Hereafter, $\langle id \rangle$, $\langle path \rangle$ and $\langle key \rangle$ allways represent pure strings.

```

__bnvs_g_IPK:w  __bnvs_g_IPK:w {<id>} {<path>} {<key>}
__bnvs_g_IP:w   __bnvs_g_IP:w  {<id>} {<path>}
__bnvs_g_I:w    __bnvs_g_I:w   {<id>}

```

Create a unique global name from the arguments.

```

670 \BNVS_new:cpn { g_IPK:w } #1 #2 #3 { g__bnvs@#1!#2/#3_tl }
671 \BNVS_new:cpn { g_IP:w } #1 #2 { g__bnvs@#1!#2_keys_seq }
672 \BNVS_new:cpn { g_I:w } #1 { g__bnvs@#1_paths_seq }

```

$\backslash g_bnvs_I_seq$ Global list of globally registered identifiers.

(End of definition for $\backslash g_bnvs_I_seq$.)

```

673 \seq_new:N \g__bnvs_I_seq

```

6.7.2 Data model

The whole data model is somehow similar to a hierarchical file system: the $\langle id \rangle$ corresponds to the drive, the $\langle path \rangle$ points to a directory, each directory may contain individual files. File named “=” contains initial data whereas file named “@” contains the static resolved data. The latter is initially undefined and becomes empty as soon as the initial data is defined or set. In order to allow index override, files respectively named “ $\langle N \rangle$ ” and “ $@ \langle N \rangle$ ” are used similarly, $\langle N \rangle$ denoting a normalized integer (no leading 0, no leading + sign). The dot (.) is used to separate components of $\langle path \rangle$.

6.7.3 Low level IPK model functions

These are **gset** and **gunset** functions. Each **gset** function must be balanced by a **gunset** function to prevent “memory leaks”.

```

__bnvs_gset:nnnn  __bnvs_gset:nnnn {<id>} {<path>} {<key>} {<def>}
__bnvs_gset:(nnnv|nnne|vnvn|nvnn|nvvn|vvnn|vvvnv)

```

Manage the storage. Keep track of all the $\langle path \rangle$ ’s for a given $\langle id \rangle$ and all the $\langle key \rangle$ ’s for a given $\langle id \rangle! \langle path \rangle$ combination.

Implementation details, next macros are defined lazily at the global level:

- $\backslash_bnvs_gset@ \langle id \rangle! \langle path \rangle:nn \{ \langle key \rangle \} \{ \langle def \rangle \}$,
- $\backslash_bnvs_gset@ \langle id \rangle:nnn \{ \langle path \rangle \} \{ \langle key \rangle \} \{ \langle def \rangle \}$,
- $\backslash g_bnvs@ \langle id \rangle!_paths_tl$, records all $\langle path \rangle$ for $\langle id \rangle$,
- $\backslash g_bnvs@ \langle id \rangle! \langle path \rangle_keys_tl$, records all $\langle key \rangle$ for $\langle id \rangle! \langle path \rangle$,
- $\backslash g_bnvs@ \langle id \rangle! \langle path \rangle_keys_tl$,

- `\g__bnvs@<id>!<path>/<key>_tl`, record the `<def>` for the `<id>!<path>/<key>` combination.

```

674 \BNVS_new:cpn { gset:Nn } #1 {
675   \tl_gclear_new:N #1
676   \tl_gset:Nn #1
677 }
678 \BNVS_new:cpn { gset_IPKV:NNw } #1 #2 #3 #4 {
679   \seq_gclear_new:N #1
680   \cs_new:Npn #2 ##1 ##2 {
681     \seq_if_in:NnF #1 { ##1 } {
682       \seq_gput_right:Nn #1 { ##1 }
683     }
684     \exp_args:Nc \__bnvs_gset:Nn {
685       \__bnvs_g_IPK:w { #3 } { #4 } { ##1 }
686     } { ##2 }
687   }
688   #2
689 }
690 \BNVS_new:cpn { gset:NNnnnn } #1 #2 #3 #4 {
691   \cs_if_exist_use:NF #1 {
692     \seq_gput_right:Nn \g__bnvs_I_seq { #3 }
693     \seq_gclear_new:N #2
694     \cs_new:Npn #1 ##1 {
695       \seq_if_in:NnF #2 { ##1 } {
696         \seq_gput_right:Nn #2 { ##1 }
697       }
698       \exp_args:Ncc \__bnvs_gset_IPKV:NNw
699       { \__bnvs_g_IP:w { #3 } { ##1 } }
700       { __bnvs_gset@#3!##1:nn } { #3 } { ##1 }
701     }
702     #1
703   }
704   { #4 }
705 }
706 \BNVS_new:cpn { gset:nnnn } #1 #2 #3 #4 {
707   \cs_if_exist_use:cF { __bnvs_gset@#1!#2:nn } {
708     \exp_args:Ncc \__bnvs_gset:NNnnnn
709     { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
710   } { #3 } { #4 }
711 }
712 \BNVS_new:cpn { gset:vnnn } {
713   \BNVS_tl_use:cv { gset:nnnn }
714 }
715 \BNVS_new:cpn { gset:nvnn } #1 {
716   \BNVS_tl_use:nv { \__bnvs_gset:nnnn { #1 } }
717 }
718 \BNVS_new:cpn { gset:vvnn } {
719   \BNVS_tl_use:Nvv \__bnvs_gset:nnnn
720 }
721 \BNVS_new:cpn { gset:nnnv } #1 #2 #3 {
722   \BNVS_tl_use:nv {
723     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
724   }

```

```

725 }
726 \BNVS_new:cpn { gset:vvnv } #1 #2 #3 {
727   \BNVS_tl_use:nv {
728     \__bnvs_gset:vvnn { #1 } { #2 } { #3 }
729   }
730 }
731 \BNVS_new:cpn { gset:nvnv } {
732   \BNVS_tl_use:Nv \__bnvs_gset:nnnv
733 }
734 \cs_generate_variant:Nn \__bnvs_gset:nnnn { nnne }

```

__bnvs_gset:nnv __bnvs_gset:nnv {<id>} {<path>} {<key>}

Syntactic sugar for __bnvs_gset:nnnv {<id>} {<path>} {<key>} {<key>}.

```

735 \BNVS_new:cpn { gset:nnv } #1 #2 #3 {
736   \__bnvs_gset:nnnv { #1 } { #2 } { #3 } { #3 }
737 }

```

__bnvs_gunset:nnn __bnvs_gunset:nnv {<id>} {<path>} {<key>}

__bnvs_gunset:vvnn

Removes the specifications for the <id>!
<path>/<key> combination.

```

738 \seq_new:N \l__bnvs_I_seq
739 \BNVS_new:cpn { seq_gremove_once:Nn } #1 #2 {
740   \seq_clear:N \l__bnvs_I_seq
741   \cs_set:Npn \BNVS_seq_gremove_Nn:n ##1 {
742     \tl_if_eq:nnTF { ##1 } { #2 } {
743       \cs_set:Npn \BNVS_seq_gremove_Nn:n #####1 {
744         \seq_put_right:Nn \l__bnvs_I_seq { #####1 }
745       }
746     } {
747       \seq_put_right:Nn \l__bnvs_I_seq { ##1 }
748     }
749   }
750   \seq_map_function:NN #1 \BNVS_seq_gremove_Nn:n
751   \seq_gset_eq:NN #1 \l__bnvs_I_seq
752 }
753 \BNVS_new:cpn { gunset_IP:Nw } #1 #2 #3 {
754   \__bnvs_seq_gremove_once:Nn #1 { #3 }
755   \seq_if_empty:NT #1 {
756     \__bnvs_seq_gremove_once:Nn \g__bnvs_I_seq { #2 }
757     \cs_undefine:c { __bnvs_gset@#2:nnn}
758   }
759 }
760 \BNVS_new:cpn { gunset:NNnnn } #1 #2 #3 #4 #5 {
761   \tl_if_exist:NT #1 {
762     \cs_undefine:N #1
763     \__bnvs_seq_gremove_once:Nn #2 { #5 }
764     \seq_if_empty:NT #2 {
765       \exp_args:Nc \__bnvs_gunset_IP:Nw { \__bnvs_g_I:w { #3 } } { #3 } { #4 }
766       \cs_undefine:c { __bnvs_gset@#3!#4:nn}
767     }
768   }
769 }
770 \BNVS_new:cpn { gunset:nnn } #1 #2 #3 {

```

```

771 \exp_args:Ncc
772 \__bnvs_gunset:NNnnn
773 { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
774 { \__bnvs_g_IP:w { #1 } { #2 } }
775 { #1 } { #2 } { #3 }
776 }
777 \BNVS_new:cpn { gunshot:vvv } {
778 \BNVS_tl_use:Nvv \__bnvs_gunset:nnn
779 }

```

```

\__bnvs_is_gset:nnnTF \__bnvs_is_gset:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}
\__bnvs_is_gset:(nvn|vvv|vxv)TF \__bnvs_is_gset:nnnTF {<id>} {<path>} {<yes:>} {<no:>}
\__bnvs_is_gset:nnTF

```

Test for the existence of some data for that $\langle id \rangle! \langle path \rangle / \langle key \rangle$ combination. The version with no $\langle key \rangle$ corresponds to key ‘=’.

```

780 \BNVS_new_conditional:cpnn { is_gset:nnn } #1 #2 #3 { T, F, TF } {
781 \tl_if_exist:cTF { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
782 \prg_return_true: \prg_return_false:
783 }
784 \BNVS_new_conditional:cpnn { is_gset:vxv } #1 #2 #3 { T, F, TF } {
785 \exp_args:NNnx \BNVS_tl_use:Nv
786 \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
787 \prg_return_true: \prg_return_false:
788 }
789 \BNVS_new_conditional:cpnn { is_gset:nvn } #1 #2 #3 { T, F, TF } {
790 \BNVS_tl_use:nv { \__bnvs_is_gset:nnnTF { #1 } } { #2 } { #3 }
791 \prg_return_true: \prg_return_false:
792 }
793 \BNVS_new_conditional:cpnn { is_gset:vvv } #1 #2 #3 { T, F, TF } {
794 \BNVS_tl_use:Nvv \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
795 \prg_return_true: \prg_return_false:
796 }
797 \BNVS_new_conditional:cpnn { is_gset:nn } #1 #2 { T, F, TF } {
798 \__bnvs_is_gset:nnnTF { #1 } { #2 } = \prg_return_true: \prg_return_false:
799 }

```

6.7.4 Loops

```

\__bnvs_foreach_I:N \__bnvs_foreach_I:N {function:n}
\__bnvs_foreach_I:n \__bnvs_foreach_I:n {<n code>}
\__bnvs_foreach_I_break: \__bnvs_foreach_I_break:
\__bnvs_foreach_I_break:n \__bnvs_foreach_I_break:n {<code>}

```

Execute the $\langle function:n \rangle$ or the $\langle n \text{ code} \rangle$ for each declared identifier. The break commands work like `\seq_map_break:` and

```

800 \BNVS_new:cpn { foreach_I:N } {
801 \seq_map_function:NN \g__bnvs_I_seq
802 }
803 \BNVS_new:cpn { foreach_I:n } {
804 \seq_map_inline:Nn \g__bnvs_I_seq
805 }

```



```

806 \cs_set_eq:NN \__bnvs_foreach_I_break: \seq_map_break:
807 \cs_set_eq:NN \__bnvs_foreach_I_break:n \seq_map_break:n

```

```

\__bnvs_foreach_P:nNTF \__bnvs_foreach_P:nNTF {<id>} <function:nn> {<yes:>} {<no:>}
\__bnvs_foreach_P:nnTF \__bnvs_foreach_P:nnTF {<id>} {<:nn code>} {<yes:>} {<no:>}

```

If $\langle id \rangle$ is a declared identifier, execute $\langle function:nn \rangle$ or $\langle code \rangle$ for each combination of $\langle id \rangle$ and its associate $\langle path \rangle$ s. Both are the arguments passed to $\langle function:nn \rangle$ or $\langle :nn code \rangle$ (through ##1 and ##2).

```

808 \BNVS_new_conditional:cpnn { foreach_P:nN } #1 #2 { T, F, TF } {
809   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
810     \seq_map_inline:cn {
811       \__bnvs_g_I:w { #1 }
812     } {
813       #2 { #1 } { ##1 }
814     }
815     \prg_return_true:
816   } {
817     \prg_return_false:
818   }
819 }
820 \BNVS_new_conditional:cpnn { foreach_P:nn } #1 #2 { T, F, TF } {
821   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
822     \cs_set:Npn \BNVS_foreach_P_nn:nn ##1 ##2 { #2 }
823     \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
824       \BNVS_foreach_P_nn:nn { #1 } { ##1 }
825     }
826     \prg_return_true:
827   } {
828     \prg_return_false:
829   }
830 }

```

```

\__bnvs_foreach_K:nnN \__bnvs_foreach_K:nnN {<id>} {<path>} <function:nnn>
\__bnvs_foreach_K:nnn \__bnvs_foreach_K:nnn {<id>} {<path>} {<:nnn code>}

```

Execute the $\langle function:nnn \rangle$ or the $\langle :nnn code \rangle$ for each of $\langle id \rangle!$ $\langle path \rangle/$ $\langle key \rangle$ combination. These are the arguments to the function or code argument.

```

831 \BNVS_new:cpn { foreach_K:NnnN } #1 #2 #3 #4 {
832   \seq_if_exist:NT #1 {
833     \seq_map_inline:Nn #1 {
834       #4 { #2 } { #3 } { ##1 }
835     }
836   }
837 }
838 \BNVS_new:cpn { foreach_K:nnN } #1 #2 #3 {
839   \exp_args:Nc \__bnvs_foreach_K:NnnN
840   { \__bnvs_g_IP:w { #1 } { #2 } }
841   { #1 } { #2 } #3
842 }
843 \BNVS_new:cpn { foreach_K:nnn } #1 #2 #3 {
844   \cs_set:Npn \BNVS_foreach_K_nnn:w ##1 ##2 ##3 { #3 }
845   \__bnvs_foreach_K:nnN { #1 } { #2 } \BNVS_foreach_K_nnn:w
846 }

```

```

\__bnvs_foreach_IP:N \__bnvs_foreach_IP:N <function:nn>
\__bnvs_foreach_IP:n \__bnvs_foreach_IP:n {<:nn code>}

```

Execute the $\langle function:nn \rangle$ or the $\langle :nn \text{ code} \rangle$ for each $\langle id \rangle! \langle path \rangle$ combination.

```

847 \BNVS_new:cpn { foreach_IP:N } #1 {
848   \__bnvs_foreach_I:n {
849     \__bnvs_foreach_P:nNT { ##1 } #1 { }
850   }
851 }

852 \BNVS_new:cpn { foreach_IP:NT } #1 #2 {
853   \__bnvs_foreach_I:n {
854     \__bnvs_foreach_P:nNT { ##1 } #1 { #2 }
855   }
856 }

857 \BNVS_new:cpn { foreach_IP:n } #1 {
858   \cs_set:Npn \BNVS_foreach_IP_n:nn ##1 ##2 { #1 }
859   \__bnvs_foreach_I:n {
860     \__bnvs_foreach_P:nNT { ##1 } \BNVS_foreach_IP_n:nn { }
861   }
862 }

```

6.7.5 Low level IP model functions

6.7.6 Low level I model functions

6.7.7 Getting

```

\__bnvs_if_get:nnncTF \__bnvs_if_get:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}
\__bnvs_if_spec:nnncTF \__bnvs_if_spec:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}
\__bnvs_if_get:nncTF \__bnvs_if_get:nncTF {<id>} {<path>} {<ans>} {<yes:>} {<no:>}
\__bnvs_if_get:vvcTF

```

The `\__bnvs_if_get:nnnc...` variant puts what was stored for $\langle id \rangle! \langle path \rangle / \langle key \rangle$ into the $\langle ans \rangle$ variable, if any, then executes $\langle yes: \rangle$. Otherwise executes $\langle no: \rangle$ without changing the contents of the $\langle ans \rangle$ tl variable.

`\__bnvs_if_get:nncTF` is a convenient shortcut for `\__bnvs_if_get:nnncTF` when $\langle key \rangle$ and $\langle ans \rangle$ happen to be the same.

The `_spec:` variant is similar except that it uses $\langle id \rangle! \langle path \rangle / \langle key \rangle$ combinations when available or $! \langle path \rangle / \langle key \rangle$ otherwise.

```

863 \BNVS_new_conditional:cpnn { if_get:nnnc } #1 #2 #3 #4 { T, F, TF } {
864   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
865     \BNVS_tl_use:Nc \cs_set_eq:Nc
866       { #4 } { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
867     \prg_return_true:
868   } {
869     \prg_return_false:
870   }
871 }

872 \BNVS_new_conditional:cpnn { if_spec:nnnc } #1 #2 #3 #4 { T, F, TF } {
873   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {

```

```

874     \tl_set_eq:cc { \BNVS_1:cn { #4 } { tl } } {
875         \__bnvs_g_IPK:w { #1 } { #2 } { #3 }
876     }
877     \prg_return_true:
878 } {
879     \tl_if_empty:nTF { #1 } {
880         \prg_return_false:
881     } {
882         \__bnvs_if_spec:nnncTF {} { #2 } { #3 } { #4 }
883         \prg_return_true: \prg_return_false:
884     }
885 }
886 }

887 \BNVS_new_conditional:cpnn { if_get:nnc } #1 #2 #3 { T, F, TF } {
888     \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #3 }
889     \prg_return_true: \prg_return_false:
890 }

891 \BNVS_new_conditional:cpnn { if_get:vvc } #1 #2 #3 { T, F, TF } {
892     \BNVS_tl_use:Nvv \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 }
893     \prg_return_true: \prg_return_false:
894 }

```

__bnvs_if_resolved:nnncTF __bnvs_if_resolved:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}

Execute <yes:> if resolution succeeded for either <id>!<<path>/<key> or the fallback !<path>/<key>, otherwise execute <no:>.

```

895 \BNVS_new_conditional:cpnn { if_resolved:nnnc } #1 #2 #3 #4 { T, F, TF } {
896     \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #4 } {
897         \__bnvs_is_will_gset:cTF { #3 }
898         \prg_return_true: \prg_return_false:
899     } {
900         \tl_if_empty:nTF { #1 } {
901             \prg_return_false:
902         } {
903             \__bnvs_if_resolved:nnncTF { #1 } { #2 } { #3 } { #4 }
904             \prg_return_true: \prg_return_false:
905         }
906     }
907 }

```

__bnvs_require:nnnc __bnvs_require:nnnc {<id>} {<path>} {<key>} {<ans>}
__bnvs_require:nnc __bnvs_require:nnc {<id>} {<path>} {<ans>}
__bnvs_require:vvc

Calls __bnvs_if_get:nnncTF, does nothing on success and, on failure, does nothing or raise when in debug mode. __bnvs_require:nnc is a shortcut for the more qualified function __bnvs_require:nnnc when <key> and <ans> happen to be identical.

```

908 \BNVS_new:cpn { require:nnnc } #1 #2 #3 #4 {
909     \__bnvs_if_get:nnncF { #1 } { #2 } { #3 } { #4 } {
910         \BNVS_fatal_unreachable:
911     }
912 }

```

<code>_bnvs_gunset:nn</code> <code>_bnvs_gunset:(nv vn vv)</code>	<code>_bnvs_gunset:nn {<id>} {<path>}</code> Removes the specifications for the $\langle id \rangle! \langle path \rangle$ and all possible keys combination.
--	--

```

913 \BNVS_new:cpn { gunshot:NNNnn } #1 #2 #3 #4 #5 {
914   \seq_map_inline:Nn #1 {
915     \\_bnvs_gunset:nnn { #4 } { #5 } { ##1 }
916   }
917   \cs_undefine:N #1
918   \\_bnvs_seq_gremove_once:Nn #2 { #5 }
919   \seq_if_empty:NT #2 {
920     \cs_undefine:N #2
921     \\_bnvs_seq_gremove_once:Nn #3 { #4 }
922     \cs_undefine:c { \\_bnvs_gset@ #4 :nnn }
923   }
924 }
925 \BNVS_new:cpn { gunshot:Nnn } #1 #2 #3 {
926   \seq_map_inline:Nn #1 {
927     \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
928       \exp_args:Nc \\_bnvs_gunset:NNNnn
929       { \\_bnvs_g_IP:w { #2 } { ##1 } }
930       #1 \g_bnvs_I_seq { #2 } { ##1 }
931       \cs_undefine:c { \\_bnvs_gset@{ #2 } { ##1 }:nn }
932     }
933   }
934 }
935 \BNVS_new:cpn { gunshot:nn } #1 #2 {
936   \exp_args:Nc
937   \\_bnvs_gunset:Nnn { \\_bnvs_g_I:w { #1 } } { #1 } { #2 }
938 }

939 \BNVS_new:cpn { gunshot:nv } #1 {
940   \BNVS_tl_use:nv { \\_bnvs_gunset:nn { #1 } }
941 }

942 \BNVS_new:cpn { gunshot:vn } {
943   \BNVS_tl_use:cv { gunshot:nn }
944 }

945 \BNVS_new:cpn { gunshot:vv } {
946   \BNVS_tl_use:Nvv \\_bnvs_gunset:nn
947 }

```

<code>_bnvs_gunset_deep:nn</code> <code>_bnvs_gunset_deep:vv</code>	<code>_bnvs_gunset_deep:nn {<id>} {<path>}</code> Removes the specifications for each existing $\langle id \rangle! \langle path \rangle. \langle subpath \rangle$ combination.
--	--

```

948 \BNVS_new:cpn { gunshot_deep:Nnn } #1 #2 #3 {
949   \seq_if_exist:NT #1 {
950     \seq_map_inline:Nn #1 {

```

#3 is exactly **##1** or start with **##1.**, which is equivalent to **#3.** starts with **##1.**:

```

951     \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
952       \__bnvs_gunset:nn { #2 } { ##1 }
953     }
954   }
955 }
956 }

957 \BNVS_new:cpn { gunshot_deep:nn } #1 #2 {
958   \exp_args:Nc \__bnvs_gunset_deep:Nnn
959   { \__bnvs_g_I:w { #1 } }
960   { #1 } { #2 }
961 }

962 \BNVS_new:cpn { gunshot_deep:vv } {
963   \BNVS_tl_use:Nvv \__bnvs_gunset_deep:nn
964 }

```

```

\__bnvs_gunset:n \__bnvs_gunset:n {<id>}
\__bnvs_gunset: \__bnvs_gunset:

```

Removes the specifications for the $\langle id \rangle$ and all possible tag combination. The second does the same for all possible $\langle id \rangle$.

```

965 \BNVS_new:cpn { gunshot:NNNn } #1 #2 #3 #4 {
966   \cs_if_exist:NT #1 {
967     \cs_undefine:N #1
968     \seq_map_inline:Nn #2 {
969       \__bnvs_gunset:nn { #4 } { ##1 }
970     }
971     \cs_undefine:N #2
972     \__bnvs_seq_gremove_once:Nn #3 { #4 }
973   }
974 }
975 \BNVS_new:cpn { gunshot:n } #1 {
976   \exp_args:Ncc \__bnvs_gunset:NNNn
977   { \__bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } }
978   \g__bnvs_I_seq { #1 }
979 }
980 \BNVS_new:cpn { gunshot: } {
981   \seq_map_inline:Nn \g__bnvs_I_seq {
982     \__bnvs_gunset:n { ##1 }
983   }
984 }

```

```

\__bnvs_VS_gunset: \__bnvs_VS_gunset:

```

Removes any value for the “V” and “S” keys and any $\langle id \rangle!$ $\langle path \rangle$ combination.

```

985 \BNVS_new:cpn { VS_gunset: } {
986   \__bnvs_foreach_IP:n {
987     \__bnvs_gunset:nnn { ##1 } { ##2 } V
988     \__bnvs_gunset:nnn { ##1 } { ##2 } S
989   }
990 }

```

6.7.8 Circular dependencies

`\_bnvs_will_gset:nnn` `\_bnvs_will_gset:nnn {⟨id⟩} {⟨path⟩} {⟨key⟩}`

To manage circular dependencies. After this command, `\_bnvs_is_will_gset:nnnTF` is true for the same arguments.

```

991 \BNVS_new:cpn { will_gset:nnn } #1 #2 #3 {
992   \_bnvs_gset:nnnn { #1 } { #2 } { #3 } { \q_no_value }
993 }

```

`\_bnvs_is_will_gset:cTF` `\_bnvs_is_will_gset:cTF {⟨in⟩} {⟨yes:⟩} {⟨no:⟩}`

To manage circular dependencies. See `\_bnvs_will_gset:nnn` above.

```

1  \_bnvs_will_gset:nnn {⟨id⟩} {⟨path⟩} {⟨key⟩}
2  ...
3  \_bnvs_if_get:nnncT {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨in⟩} {
4    ...
5    \_bnvs_is_will_gset:cTF
6      {⟨in⟩}
7      {⟨yes:⟩}
8      {⟨no:⟩}
9    ...
10 }

```

```

994 \BNVS_new_conditional:cpnn { is_will_gset:c } #1 { T, F, TF } {
995   \BNVS_tl_use:Nv \quark_if_no_value:nTF { #1 } {
996     \prg_return_true:
997   } {
998     \prg_return_false:
999   }
1000 }

```

6.7.9 Spring cleaning

`\_bnvs_gclear:nn` `\_bnvs_gclear:nn {⟨id⟩} {⟨path⟩}`
`\_bnvs_gclear:nv` `\_bnvs_gclear:n {⟨id⟩}`
`\_bnvs_gclear:` `\_bnvs_gclear:`
`\_bnvs_greset:nn` `\_bnvs_greset:nn {⟨id⟩} {⟨path⟩}`
`\_bnvs_greset:nv` `\_bnvs_greset:n {⟨id⟩}`
`\_bnvs_greset:` `\_bnvs_greset:`

The first ones clear out every data stored for `⟨id⟩!⟨path⟩`, including deeper and registration data. When `⟨path⟩` is omitted, any known path is considered. When `⟨id⟩` is omitted, any known id is considered. The second one only clears out the data created on the fly after the initialization step. The initial data (for key =) is left untouched.

```

1001 \BNVS_new:cpn { greset:nn } #1 #2 {

```

```

1002  \__bnvs_foreach_P:nnT { #1 } {
1003    \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1004      \__bnvs_foreach_K:nnn { #1 } { ##2 } {
1005        \tl_if_in:nnF { \q_bnvs ####3 } { \q_bnvs = } {
1006          \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1007        }
1008      }
1009    }
1010  } { }
1011 }
  Clears out every data associated to <id>!<path>, and any <id>!<path>.<subpath>.
1012 \BNVS_new:cpn { gclear:nn } #1 #2 {
1013   \__bnvs_foreach_P:nnT { #1 } {
1014     \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1015       \__bnvs_foreach_K:nnn { #1 } { ##2 } {
1016         \tl_if_in:nnTF { \q_bnvs ####3 } { \q_bnvs = } {
1017           \__bnvs_gset:nnnn { #1 } { ##2 } { ####3 } { 0 }
1018         } {
1019           \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1020         }
1021       }
1022     }
1023   } { }
1024 }
1025 \BNVS_new:cpn { gclear:n } #1 {
1026   \__bnvs_foreach_P:nNT { #1 } \__bnvs_gclear:nn {}
1027 }
1028 \BNVS_new:cpn { gclear: } {
1029   \__bnvs_foreach_IP:N \__bnvs_gclear:nn
1030 }
1031 \BNVS_new:cpn { greset:n } #1 {
1032   \__bnvs_foreach_P:nNT { #1 } \__bnvs_greset:nn {}
1033 }
1034 \BNVS_new:cpn { greset: } {
1035   \__bnvs_foreach_IP:N \__bnvs_greset:nn
1036 }

```

__bnvs_gclear_resolved:nn __bnvs_gclear_resolved:nn {<id>} {<path>}

__bnvs_gclear_resolved:nv Clear the resolved data, aka any key that does not start with “=”.

```

1037 \BNVS_new:cpn { gclear_resolved:nn } #1 #2 {
1038   \__bnvs_foreach_IP:nnn { #1 } { #2 } {
1039     \tl_if_in:nnF { @ ##1 } { @ = } {
1040       \__bnvs_gunset:nnn { #1 } { #2 } { ##1 }
1041     }
1042   }
1043 }
1044 \BNVS_new:cpn { gclear_resolved:nv } #1 {
1045   \BNVS_tl_use:nv {
1046     \__bnvs_gclear_resolved:nn { #1 }
1047   }
1048 }

```

6.7.10 The provide mode

`\l__bnvs_provide_bool` Manage the provide mode.

```
1049 \bool_new:N \l__bnvs_provide_bool
      (End of definition for \l__bnvs_provide_bool.)
```

```
1050 \BNVS_new:cpn { provide_ON: } {
1051   \__bnvs_set_true:c { provide }
1052 }
1053 \BNVS_new:cpn { provide_OFF: } {
1054   \__bnvs_set_false:c { provide }
1055 }
1056 \__bnvs_provide_OFF:
```

```
\__bnvs_if_should_provide:nnnTF \__bnvs_if_should_provide:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}
\__bnvs_if_should_provide:nnTF \__bnvs_if_should_provide:nnTF {<id>} {<path>} {<yes:>} {<no:>}
```

Execute `<yes:>` when in provide mode and `<id>!``<path>/``<key>` is known, `<no:>` otherwise.
When not specified, `<key>` is “=”.

```
1057 \BNVS_new_conditional:cpnn { if_should_provide:nnn } #1 #2 #3 { T, F, TF } {
1058   \__bnvs_if:cTF { provide } {
1059     \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
1060       \prg_return_false:
1061     } {
1062       \prg_return_true:
1063     }
1064   } {
1065     \prg_return_true:
1066   }
1067 }
1068 \BNVS_new_conditional:cpnn { if_should_provide:nn } #1 #2 { T, F, TF } {
1069   \__bnvs_if_should_provide:nnnTF { #1 } { #2 } =
1070   \prg_return_true: \prg_return_false:
1071 }
```

```
\__bnvs_is_gset_and_provide:nnnTF \__bnvs_is_gset_and_provide:nnnTF {<id>} {<path>} {<key>}
\__bnvs_is_gset_and_provide:(nvn|vvn)TF {<yes:>} {<no:>}
\__bnvs_is_gset_and_provide:nnTF \__bnvs_is_gset_and_provide:nnTF {<id>} {<path>} {<yes:>}
\__bnvs_is_gset_and_provide:nnTF {<no:>}
```

Execute `<yes:>` when in provide mode and `<id>!``<path>/``<key>` is known, `<no:>` otherwise.
When not specified, `<key>` is “=”.

```
1072 \BNVS_new_conditional:cpnn { is_gset_and_provide:nnn }
1073   #1 #2 #3 { T, F, TF } {
1074   \__bnvs_if:cTF { provide } {
1075     \if_cs_exist:w \__bnvs_g_IPK:w { #1 } { #2 } { #3 } \cs_end:
1076     \prg_return_true:
1077   } \else:
1078     \prg_return_false:
1079   \fi:
1080 }
```



```

1081     \prg_return_false:
1082   }
1083 }
1084 \BNVS_new_conditional:cpnn { is_gset_and_provide:nvn }
1085     #1 #2 #3 { T, F, TF } {
1086   \BNVS_tl_use:nv {
1087     \__bnvs_is_gset_and_provide:nnnTF { #1 }
1088   } { #2 } { #3 }
1089   \prg_return_true: \prg_return_false:
1090 }
1091 \BNVS_new_conditional:cpnn { is_gset_and_provide:vvv }
1092     #1 #2 #3 { T, F, TF } {
1093   \BNVS_tl_use:Nvv \__bnvs_is_gset_and_provide:nnnTF { #1 } { #2 } { #3 }
1094   \prg_return_true: \prg_return_false:
1095 }

```

```

\__bnvs_gprovide:TnnnnF \__bnvs_gprovide:TnnnnF {<pre code>} {<id>} {<path>} {<key>} {<value>} {<no:>}
\__bnvs_gprovide:TvnnnF
\__bnvs_gprovide:TvvnnF

```

Execute *<no:>* exclusively when not in provide mode. Does nothing when something was set for the *<id>!**<path>/<key>* combination. Execute *<pre code>* before setting.

```

1096 \BNVS_new:cpn { gprovide:TnnnnF } #1 #2 #3 #4 #5 {
1097   \__bnvs_if:cTF { provide } {
1098     \__bnvs_is_gset:nnnF { #2 } { #3 } { #4 } {
1099       #1
1100       \__bnvs_gset:nnnn { #2 } { #3 } { #4 } { #5 }
1101     }
1102   }
1103 }
1104 \BNVS_new:cpn { gprovide:TvnnnF } #1 {
1105   \BNVS_tl_use:nv { \__bnvs_gprovide:TnnnnF { #1 } }
1106 }
1107 \BNVS_new:cpn { gprovide:TvvnnF } #1 {
1108   \BNVS_tl_use:nvv { \__bnvs_gprovide:TnnnnF { #1 } }
1109 }

```

6.7.11 Initialize

Here, *<simple definition>* contains no *<key>-<value>* material, even implicit.

```

\__bnvs_ginit:nnn \__bnvs_ginit:nnn {<id>} {<path>} {<simple definition>}
\__bnvs_ginit:(nnv|vvv)

```

This function supports provide mode by calling `\__bnvs_is_gset_and_provide:nnnTF`. Somehow a shortcut to `\__bnvs_gset:nnnn` with key “=”. If *<path>* has more than one component, each intermediate path is modified or defined appropriately.

```

1110 \BNVS_new:cpn { ginit:nnn } #1 #2 #3 {
1111   \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
1112     \__bnvs_greset:nn { #1 } { #2 }
1113     \__bnvs_gset:nnnn { #1 } { #2 } = { #3 }
1114   }
1115 }
1116 \BNVS_new:cpn { ginit:nnv } #1 #2 {
1117   \BNVS_tl_use:nv { \__bnvs_ginit:nnn { #1 } { #2 } }
1118 }

```

```

1119 \BNVS_new:cpn { ginit:vvv } {
1120   \BNVS_tl_use:Nvv \__bnvs_ginit:nnn
1121 }

```

__bnvs_ginit_ITPD:w __bnvs_ginit_ITPD:w {<id>} {<path>} {<index>} {<simple definition>}

This functions supports provide mode by calling `\__bnvs_is_gset_and_provide:nnnTF`. Somehow a shortcut to `\__bnvs_gset:nnnn` with key <index>. When <index> is provided, it is already normalized. If <id>!
<path>/= is not set, it is initialized from <definition> shifted appropriately relative to <index>.

```

1122 \BNVS_new:cpn { ginit_ITPD:w } #1 #2 #3 #4 {
1123   \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } { #3 } {
1124     \__bnvs_gunset:nnn { #1 } { #2 } { #3 }
1125     \__bnvs_gset:nnnn { #1 } { #2 } { =#3 } { #4 }
1126   }
1127   \__bnvs_is_gset:nnnF { #1 } { #2 } = {
1128     \__bnvs_gset:nnne { #1 } { #2 } = {
1129       \exp_not:n { #4 } - \int_eval:n { #3 - 1 }
1130     }
1131   }
1132 }

```

__bnvs_gdef_IPKD:w __bnvs_gdef_IPKD:w {<id>} {<path>} {<key>} {<simple definition>}

This function calls `\__bnvs_gdef:nnn` with <path>.
<key> as second argument.

```

1133 \BNVS_new:cpn { gdef_IPKD:w } #1 #2 #3 #4 {
1134   \__bnvs_gdef:nnn { #1 } { #2.#3 } { #4 }
1135 }

```

6.7.12 Provide mode

__bnvs_is_provided:nnnTF __bnvs_is_provided:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}

If <id>!
<path>/<key> has been successfully resolved and not unset since then, execute <yes:>. Otherwise execute <no:>.

```

1136 \tl_new:N \l__bnvs_is_provided_tl
1137 \BNVS_new_conditional:cpnn { is_provided:nnn } #1 #2 #3 { T, F, TF } {
1138   \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { is_provided } {
1139     \__bnvs_is_will_gset:cTF { is_provided } {
1140       \prg_return_false:
1141     } {
1142       \prg_return_true:
1143     }
1144   } {
1145     \prg_return_false:
1146   }
1147 }

```

6.8 Regular expressions

`\c__bnvs_S_regex` This regular expression is used for both short names and dot path components. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit. It must contain one uppercase letter.

```
1148 \regex_const:Nn \c__bnvs_S_regex {
1149   (?:[a-z_][a-z0-9_]*)?[A-Z][[:alnum:]]_*
1150 }
```

(End of definition for \c__bnvs_S_regex.)

`\c__bnvs_R_regex` A possibly empty list of `.<short name>` items representing a path.

```
1151 \regex_const:Nn \c__bnvs_R_regex {
1152   (?:\. \ur{c__bnvs_S_regex})*
1153 }
```

(End of definition for \c__bnvs_R_regex.)

`\c__bnvs_U_regex` An `<attribute>`, no uppercase variable.

```
1154 \regex_const:Nn \c__bnvs_U_regex {
1155   [a-z_][a-z0-9_]_*
1156 }
```

(End of definition for \c__bnvs_U_regex.)

`\c__bnvs_O_regex` A possibly empty list of `<attributes>` items representing a path.

```
1157 \regex_const:Nn \c__bnvs_O_regex {
1158   (?:\. \ur{c__bnvs_U_regex})*
1159 }
```

(End of definition for \c__bnvs_O_regex.)

`\c__bnvs_A_N_Z_regex` Signed integer with two capture groups.

(End of definition for \c__bnvs_A_N_Z_regex.)

```
1160 \regex_const:Nn \c__bnvs_A_N_Z_regex {
1161   \A \. \s* [-+\s]* \d+ \Z
1162 }
```

`\c__bnvs_A_index_Z_regex` Signed integer with no capture groups.

(End of definition for \c__bnvs_A_index_Z_regex.)

```
1163 \regex_const:Nn \c__bnvs_A_index_Z_regex { \A [-+\s]* \d + \Z }
```

`\c__bnvs_A_reserved_Z_regex` Reserved words.

(End of definition for \c__bnvs_A_reserved_Z_regex.)

```
1164 \regex_const:Nn \c__bnvs_A_reserved_Z_regex {
1165   \A*[a-z][_a-z0-9]*\Z
1166 }
```

`\c__bnvs_A_IKPN_Z_regex` Matches the whole string, split into `<id>` and `<path>`.

(End of definition for \c__bnvs_A_IKPN_Z_regex.)

1: The full match,

```
1167 \regex_const:Nn \c__bnvs_A_IKPN_Z_regex {
```

2,3: the optional frame `<id>` and `!`

```
1168 \A (?:( \ur{c__bnvs_S_regex} )? (!) )?
```

4: The tag (short name and dotted path)

```
1169 ( \ur{c__bnvs_S_regex} \ur{c__bnvs_R_regex} ? )
```

5: a signed number that will be normalized.

```
1170 (?:( \. \s* ([-+]? \s* \d+ ) )? \Z
1171 }
```

`\c__bnvs_A_IKSRN_Z_regex` Matches the whole string, split into $\langle id \rangle$ and $\langle tag \rangle$.
(End of definition for `\c__bnvs_A_IKSRN_Z_regex`.)

1: The full match,

1172 `\regex_const:Nn \c__bnvs_A_IKSRN_Z_regex {`

2,3: the optional frame $\langle id \rangle$ and !

1173 `\A(?: ( \ur{c__bnvs_S_regex} )? (!) )?`

4: The short name

1174 `( \ur{c__bnvs_S_regex} )`

5: The dotted path

1175 `( \ur{c__bnvs_R_regex} )?`

6: a signed number that will be normalized.

1176 `(?: \s* \. \s* ([-+]? \s* \d+ ) )? \Z`
1177 `}`

`\c__bnvs_A_SR_Z_regex` Matches the whole string.
(End of definition for `\c__bnvs_A_SR_Z_regex`.)

1178 `\regex_const:Nn \c__bnvs_A_SR_Z_regex {`

1: The full match,

2: the frame $\langle id \rangle$

1179 `\A ( \ur{c__bnvs_S_regex} | [-+]? \d+ )`

3: The dotted path.

1180 `( (?: \. \ur{c__bnvs_S_regex} | \. [-+]? \d+ )* ) \Z`
1181 `}`

`\c__bnvs_A_P_Z_regex` Matches the whole string.
(End of definition for `\c__bnvs_A_P_Z_regex`.)

1182 `\regex_const:Nn \c__bnvs_A_P_Z_regex {`
1183 `\A \ur{c__bnvs_S_regex} \ur{c__bnvs_R_regex} \Z`
1184 `}`

`\c__bnvs_colons_regex` For ranges defined by a colon syntax. One catching group for more than one colon.

1185 `\regex_const:Nn \c__bnvs_colons_regex { :(:+)? }`
(End of definition for `\c__bnvs_colons_regex`.)

`\c__bnvs_A_VAZLLZZL_Z_regex` Used to parse named overlay specifications. V, A:Z, A::L on one side, :Z, :Z::L and ::L:Z on the other sides. Next are the capture groups. The first one is for the whole match.
(End of definition for `\c__bnvs_A_VAZLLZZL_Z_regex`.)

1186 `\regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {`
1187 `\A \s* (?:`

• $2 \rightarrow V$
1188 `( [^:]+? )`

• $3, 4, 5 \rightarrow A : Z?$ or $A :: L?$
1189 `| (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )`

• $6, 7 \rightarrow ::(L:Z)?$

```

1190 | (? : : \s* (? : ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
      • 8, 9 → :(Z::L)?
1191 | (? : : \s* (? : ( [^:]+? ) \s* : : \s* ( [^:]*? ) )? )
1192 )
1193 \s* \Z
1194 }

```

6.9 End group setters

```

1195 \cs_new:Npn \BNVS_after_end_tl_put_right:cv #1 {
1196   \BNVS_after_end:n {
1197     \__bnvs_tl_put_right:cn { #1 }
1198   }
1199   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1200 }
1201 \cs_new:Npn \BNVS_end_tl_put_right:cv #1 #2 {
1202   \BNVS_after_end_tl_put_right:cv { #1 } { #2 }
1203   \BNVS_end:
1204 }
1205 \cs_new:Npn \BNVS_after_end_tl_gset:nnnv #1 #2 #3 {
1206   \BNVS_after_end:n {
1207     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
1208   }
1209   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1210 }
1211 \cs_new:Npn \BNVS_end_tl_gset:nnnv #1 #2 #3 #4 {
1212   \BNVS_after_end_tl_gset:nnnv { #1 } { #2 } { #3 } { #4 }
1213   \BNVS_end:
1214 }
1215 \cs_new:Npn \BNVS_after_end_tl_set:cv #1 {
1216   \BNVS_after_end:n {
1217     \__bnvs_tl_set:cn { #1 }
1218   }
1219   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1220 }
1221 \cs_new:Npn \BNVS_end_tl_set:cv #1 #2 {
1222   \BNVS_after_end_tl_set:cv { #1 } { #2 }
1223   \BNVS_end:
1224 }
1225 \cs_new:Npn \BNVS_tl_set:ncv #1 #2 {
1226   \BNVS_tl_use:nv {
1227     #1
1228     \__bnvs_tl_set:cn { #2 }
1229   }
1230 }
1231 \cs_new:Npn \BNVS_tl_set:ncvcv #1 #2 #3 #4 {
1232   \BNVS_tl_set:ncv {
1233     \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1234   } { #4 }
1235 }

```

```

1236 \cs_new:Npn \BNVS_tl_set:ncvcvcv #1 #2 #3 #4 #5 #6 {
1237   \BNVS_tl_set:ncv {
1238     \BNVS_tl_set:ncv {
1239       \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1240     } { #4 } { #5 }
1241   } { #6 }
1242 }
1243 \cs_new:Npn \BNVS_after_end_int_set:cv #1 {
1244   \BNVS_after_end:n {
1245     \__bnvs_int_set:cn { #1 }
1246   }
1247   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1248 }
1249 \cs_new:Npn \BNVS_end_int_set:cv #1 #2 {
1250   \BNVS_after_end_int_set:cv { #1 } { #2 }
1251   \BNVS_end:
1252 }

```

6.10 beamer.cls interface

Work in progress.

```

1253 \RequirePackage{keyval}
1254 \define@key{beamerframe}{beanoves~id}[]{}
1255   \tl_set:Nx \l__bnvs_J_tl { #1 }
1256 }
1257 \bool_new:N \l__bnvs_in_frame_bool
1258 \bool_set_false:N \l__bnvs_in_frame_bool
1259 \AddToHook{env/beamer@frameslide/before}{
1260   \__bnvs_greset_call:
1261   \__bnvs_VS_gunset:
1262   \__bnvs_set_true:c { in_frame }
1263 }
1264 \AddToHook{env/beamer@frameslide/after}{
1265   \__bnvs_set_false:c { in_frame }
1266 }

```

6.11 Utilities

6.11.1 Split utilities

```

\__bnvs_if_regex_split:cncTF \__bnvs_if_regex_split:cncTF {<regex core>} {<expression>} {<seq core>}
\__bnvs_if_regex_split:cnTF {<yes:>} {<no:>}
\__bnvs_if_regex_split:cnTF {<regex core>} {<expression>} {<yes:>} {<no:>}

```

These are shortcuts to `\regex_split:NnNTF` with the `split` sequence as last `N` argument.

```

1267 \BNVS_new_conditional:cpnn { if_regex_split:cnc } #1 #2 #3 { T, F, TF } {
1268   \BNVS_seq_use:nc {
1269     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1270   } { #3 } \prg_return_true: \prg_return_false:
1271 }

```

```

1272 \BNVS_new_conditional:cpnn { if_regex_split:cn } #1 #2 { T, F, TF } {
1273   \BNVS_seq_use:nc {
1274     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1275   } { split } \prg_return_true: \prg_return_false:
1276 }

```

<pre> __bnvs_split_pop: __bnvs_split_pop:c __bnvs_split_pop:cc __bnvs_split_pop:ccc __bnvs_split_pop:cccc __bnvs_split_pop:ccccc </pre>	<pre> __bnvs_split_pop: __bnvs_split_pop:c {<var₁>} __bnvs_split_pop:cc {<var₁>} {<var₂>} __bnvs_split_pop:ccc {<var₁>} {<var₂>} {<var₃>} __bnvs_split_pop:cccc {<var₁>} {<var₂>} {<var₃>} {<var₄>} __bnvs_split_pop:ccccc {<var₁>} {<var₂>} {<var₃>} {<var₄>} {<var₅>} </pre>
---	---

```

1277 \tl_new:N \l__bnvs_split_pop_tl
1278 \BNVS_new:cpn { split_pop: } {
1279   \seq_pop_left:NN \l__bnvs_split_seq \l__bnvs_split_pop_tl
1280 }
1281 \BNVS_new:cpn { split_pop:c } #1 {
1282   \BNVS_tl_use:nc {
1283     \seq_pop_left:NN \l__bnvs_split_seq
1284   } { #1 }
1285 }
1286 \BNVS_new:cpn { split_pop:cc } #1 {
1287   \__bnvs_split_pop:c { #1 }
1288   \__bnvs_split_pop:c
1289 }
1290 \BNVS_new:cpn { split_pop:ccc } #1 {
1291   \__bnvs_split_pop:c { #1 }
1292   \__bnvs_split_pop:cc
1293 }
1294 \BNVS_new:cpn { split_pop:cccc } #1 {
1295   \__bnvs_split_pop:c { #1 }
1296   \__bnvs_split_pop:ccc
1297 }
1298 \BNVS_new:cpn { split_pop:ccccc } #1 {
1299   \__bnvs_split_pop:c { #1 }
1300   \__bnvs_split_pop:cccc
1301 }
1302 \BNVS_new_conditional:cpnn { split_if_pop_left:c } #1 { T, F, TF } {
1303   \__bnvs_seq_pop_left:ccTF { split } { #1 } {
1304     \prg_return_true:
1305   } {
1306     \prg_return_false:
1307   }
1308 }

```

6.11.2 Match utilities

```

__bnvs_match_if_once:NnTF \__bnvs_match_if_once:NnTF <regex variable> {(expression)} {(yes:)} {(no:)}
__bnvs_match_if_once:NvTF \__bnvs_match_if_once:nnTF {(regex)} {(expression)} {(yes:)} {(no:)}
__bnvs_match_if_once:nnTF

```

These are shortcuts to

- \regex_match_if_once:NnNTF with the match sequence as N argument
- \regex_match_if_once:nnNTF with the match sequence as N argument
- \regex_split:NnNTF with the split sequence as last N argument

```

1309 \BNVS_new_conditional:cpnn { if_extract_once:Ncn } #1 #2 #3 { T, F, TF } {
1310   \BNVS_use:ncn {
1311     \regex_extract_once:NnNTF #1 { #3 }
1312   } { #2 } { seq } \prg_return_true: \prg_return_false:
1313 }
\l__bnvs_match_seq Local storage for the match result.
                    (End of definition for \l__bnvs_match_seq.)
1314 \BNVS_new_conditional:cpnn { match_if_once:Nn } #1 #2 { T, F, TF } {
1315   \BNVS_use:ncn {
1316     \regex_extract_once:NnNTF #1 { #2 }
1317   } { match } { seq } \prg_return_true: \prg_return_false:
1318 }
1319 \BNVS_new_conditional:cpnn { if_extract_once:Ncv } #1 #2 #3 { T, F, TF } {
1320   \BNVS_seq_use:nc {
1321     \BNVS_tl_use:nv {
1322       \regex_extract_once:NnNTF #1
1323     } { #3 }
1324   } { #2 } \prg_return_true: \prg_return_false:
1325 }
1326 \BNVS_new_conditional:cpnn { match_if_once:Nv } #1 #2 { T, F, TF } {
1327   \BNVS_seq_use:nc {
1328     \BNVS_tl_use:nv {
1329       \regex_extract_once:NnNTF #1
1330     } { #2 }
1331   } { match } \prg_return_true: \prg_return_false:
1332 }
1333 \BNVS_new_conditional:cpnn { match_if_once:nn } #1 #2 { T, F, TF } {
1334   \BNVS_seq_use:nc {
1335     \regex_extract_once:nnNTF { #1 } { #2 }
1336   } { match } \prg_return_true: \prg_return_false:
1337 }
1338 \BNVS_new_conditional:cpnn { match_if_pop_left:c } #1 { T, F, TF } {
1339   \BNVS_tl_use:nc {
1340     \BNVS_seq_use:Nc \seq_pop_left:NNTF { match }
1341   } { #1 } {
1342     \prg_return_true:
1343   } {
1344     \prg_return_false:
1345   }
1346 }

```

```

\__bnvs_match_pop:      \__bnvs_match_pop:
\__bnvs_match_pop:c      \__bnvs_match_pop:c      {<var1>}
\__bnvs_match_pop:cc      \__bnvs_match_pop:cc      {<var1>} {<var2>}
\__bnvs_match_pop:ccc      \__bnvs_match_pop:ccc      {<var1>} {<var2>} {<var3>}
\__bnvs_match_pop:cccc      \__bnvs_match_pop:cccc      {<var1>} {<var2>} {<var3>} {<var4>}
\__bnvs_match_pop:ccccc      \__bnvs_match_pop:ccccc      {<var1>} {<var2>} {<var3>} {<var4>} {<var5>}

```

```

1347 \tl_new:N \l__bnvs_match_pop_tl
1348 \BNVS_new:cpn { match_pop: } {
1349   \seq_pop_left:NN \l__bnvs_match_seq \l__bnvs_match_pop_tl
1350 }
1351 \BNVS_new:cpn { match_pop:c } #1 {
1352   \BNVS_tl_use:nc {
1353     \seq_pop_left:NN \l__bnvs_match_seq
1354   } { #1 }
1355 }
1356 \BNVS_new:cpn { match_pop:cc } #1 {
1357   \__bnvs_match_pop:c { #1 }
1358   \__bnvs_match_pop:c
1359 }
1360 \BNVS_new:cpn { match_pop:ccc } #1 {
1361   \__bnvs_match_pop:c { #1 }
1362   \__bnvs_match_pop:cc
1363 }
1364 \BNVS_new:cpn { match_pop:cccc } #1 {
1365   \__bnvs_match_pop:c { #1 }
1366   \__bnvs_match_pop:ccc
1367 }
1368 \BNVS_new:cpn { match_pop:ccccc } #1 {
1369   \__bnvs_match_pop:c { #1 }
1370   \__bnvs_match_pop:cccc
1371 }

```

\c__bnvs_A_IKSR_Z_regex A qualified dotted name is the qualified name of an overlay set possibly followed by a dotted path. Matches the whole string. Four capture groups.
(End of definition for \c__bnvs_A_IKSR_Z_regex.)

```

1372 \regex_const:Nn \c__bnvs_A_IKSR_Z_regex {
    1: the frame <id>
    2: the exclamation mark
1373   \A(?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?
    3: The short name.
1374   ( \ur{c__bnvs_S_regex} )
    4: the remaining part of the path, if any.
1375   ( \ur{c__bnvs_R_regex} ) \Z
1376 }

```

```

\__bnvs_normalize:nTF \__bnvs_normalize:nTF {<var>} {<yes:n>} {<no:n>}

```

If {<var>} contains a decimal integer annotation, normalize it (for example remove leading zeros) and call {<yes:n>} with it.

```

1377 \BNVS_new:cpn { normalize:nTF } #1 #2 #3 {
1378   \__bnvs_if_index:nTF { #1 } {
1379     \cs_set:Npn \BNVS:n #####1 { #2 }
1380     \exp_args:Ne \BNVS:n {
1381       \int_value:w #1 \exp_stop_f:
1382     }
1383   } {
1384     \cs_set:Npn \BNVS:n #####1 { #3 }
1385     \BNVS:n { #1 }
1386   }
1387 }

```

__bnvs_normalize_S:c **__bnvs_normalize_S:c {<var>}**

If <var> contains a decimal integer annotation, normalize it (for example remove leading zeros).

```

1388 \BNVS_new:cpn { normalize_S:c } #1 {
1389   \BNVS_tl_use:Nv \__bnvs_normalize:nTF { #1 } {
1390     \__bnvs_tl_set:cn { #1 } { #####1 }
1391   } {}
1392 }

```

__bnvs_map:nnnn **__bnvs_map:nnnn {<tag>} {<S code:n>} {<R code:n>} {<no:n>}**

If the <tag> is not empty, execute <S code:n> for the first component and then apply <R code:n> to each next component. Otherwise, execute <no:n> with the <tag>.

```

1393 \BNVS_new:cpn { map_R:nnnn } #1 #2 #3 #4 {
1394   \tl_if_empty:nTF { #1 } {
1395     \cs_set:Npn \BNVS:n ##1 { #4 }
1396     \BNVS:n { #1 }
1397   } {
1398     \__bnvs_seq_set_split:cn R { . } { #1 }
1399     \__bnvs_seq_pop_left:cc R R
1400     \cs_set:Npn \BNVS:n ##1 { #2 }
1401     \BNVS_tl_use:Nv \BNVS:n R
1402     \__bnvs_seq_map_inline:cn R { #3 }
1403   }
1404 }

```

__bnvs_normalize_R:c **__bnvs_normalize_R:c {<var>}**

If <var> contains a component in decimal integer annotation, normalize it (for example remove leading zeros) and replace it in <var>.

```

1405 \BNVS_new:cpn { normalize_R:c } #1 {
1406   \BNVS_tl_use:Nv \__bnvs_map_R:nnnn { #1 } {
1407     \__bnvs_tl_clear:c { #1 }
1408   } {
1409     \__bnvs_normalize:nTF { ##1 } {
1410       \__bnvs_tl_put_right:cn { #1 } { .#####1 }
1411     } {
1412       \__bnvs_tl_put_right:cn { #1 } { .#####1 }
1413     }
1414   } {}
1415 }

```

```

__bnvs_if_IT:nTF __bnvs_if_IT:nTF {<name>} {<yes:>} {<no:>}
__bnvs_if_IT:nnTF {<root>} {<relative>} {<yes:>} {<no:>}

```

If *<name>* is a reference, put the frame id it defines, or the current frame id, into I, the short name into S, the dotted path into R, the tag into T, then execute *<yes:>*. Any component that is an integer is normalized. Otherwise execute *<no:>*.

The J variable is updated.

```

1416 \BNVS_new_conditional:cpnn { if_IT:n } #1 { T, F, TF } {
1417   \BNVS_begin:
1418   __bnvs_match_if_once:NnTF \c__bnvs_A_IKSR_Z_regex { #1 } {
1419     __bnvs_match_pop:
1420     __bnvs_match_pop:cccc IKSR
1421     \BNVS_tl_use:nv { \__bnvs_match_if_once:nnT { \A [a-z]+ \Z } } S {
1422       \BNVS_error:n { Unsupported~lowercase~only~keys. }
1423     }
1424     __bnvs_normalize_S:c S
1425     __bnvs_normalize_R:c R
1426     \cs_set:Npn \BNVS_if_IT_nTF:nnn ##1 ##2 ##3 {
1427       \BNVS_end:
1428       __bnvs_tl_set:cn I { ##1 }
1429       __bnvs_tl_set:cn S { ##2 }
1430       __bnvs_tl_set:cn R { ##3 }
1431     }
1432     __bnvs_tl_if_empty:cTF K {
1433       \BNVS_tl_use:Nvvv \BNVS_if_IT_nTF:nnn J
1434     } {
1435       \BNVS_tl_use:Nvvv \BNVS_if_IT_nTF:nnn I
1436     } S R
1437     __bnvs_tl_set:cv T R
1438     __bnvs_tl_put_left:cv T S
1439     __bnvs_tl_set:cv J I
1440     \prg_return_true:
1441   } {
1442     \BNVS_end:
1443     \prg_return_false:
1444   }
1445 }

```

```

__bnvs_if_ITN:nTF __bnvs_if_ITN:nTF {<name>} {<yes:>} {<no:>}

```

Called from *__bnvs_root_parsed:n* and *__bnvs_root_parsed:nn*. If *<name>* is a reference, put the frame id it defines, or the current frame id (the value of J), into I, the tag into T and the normalized number index into N, if any, then execute *<yes:>*. Otherwise execute *<no:>*.

The J variable is updated.

```

1446 \BNVS_new_conditional:cpnn { if_ITN:n } #1 { T, F, TF } {
1447   \BNVS_begin:
1448   __bnvs_match_if_once:NnTF \c__bnvs_A_IKPN_Z_regex { #1 } {
1449     __bnvs_match_pop:
1450     __bnvs_match_pop:cccc IKTN
1451     \cs_set:Npn \BNVS_if_ITN:w ##1 ##2 ##3 {
1452       \BNVS_end:
1453       __bnvs_tl_set:cn I { ##1 }
1454       __bnvs_tl_set:cn T { ##2 }

```

```

1455     \__bnvs_tl_set:cn N { ##3 }
1456     \__bnvs_normalize_S:c N
1457 }
1458 \__bnvs_tl_if_empty:cTF K {
1459     \BNVS_tl_use:Nvvv \BNVS_if_ITN:w J
1460 } {
1461     \BNVS_tl_use:Nvvv \BNVS_if_ITN:w I
1462 } T N
1463 \__bnvs_tl_set:cv J I
1464 \prg_return_true:
1465 } {
1466     \BNVS_end:
1467     \prg_return_false:
1468 }
1469 }
1470 \BNVS_undefine:c { if_ITN:n }

```

6.11.3 Path utilities

`\c__bnvs_A_PN_Z_regex` Matches the whole string.

(End of definition for `\c__bnvs_A_PN_Z_regex`.)

```

1471 \regex_const:Nn \c__bnvs_A_PN_Z_regex {
    1: The full match,
    2: the path
1472     \A ( \ur{c__bnvs_S_regex} \ur{c__bnvs_R_regex} )
    3: The sign and the value of the trailing digits component.
1473     (?: \. \s * ( [-+]? \s* \d+ ) )? \Z
1474 }

```

`\__bnvs_if_PN:nTTF` `\__bnvs_if_PN:nTTF {<path>} {<yes:n>} {<yes:nn>} {<no:>}`

If `<path>` is a correct path with no trailing number, execute `<yes:n>` with the whole `<path>` as argument. If `<path>` is a correct path with a trailing number, executes `<yes:nn>` with the whole `<path>` but its last component as first argument and the normalized number as second arguments. If `<path>` is not a correct path, execute `<no:>`.

```

1475 \BNVS_new:cpn { if_PN:nTTF } #1 #2 #3 #4 {
1476     \BNVS_begin:
1477     \__bnvs_match_if_once:NnTF \c__bnvs_A_PN_Z_regex { #1 } {
1478         \__bnvs_match_pop:
1479         \__bnvs_match_pop:cc PN
1480         \__bnvs_tl_if_empty:cTF { N } {
1481             \cs_set:Npn \BNVS_if_path_nTTF:w ##1 { \BNVS_end: #2 }
1482             \BNVS_tl_use:Nv \BNVS_if_path_nTTF:w P
1483         } {
1484             \__bnvs_normalize_S:c N
1485             \cs_set:Npn \BNVS_if_path_nTTF:w ##1 ##2 { \BNVS_end: #3 }
1486             \BNVS_tl_use:Nvv \BNVS_if_path_nTTF:w P N
1487         }
1488     } {
1489         \BNVS_end:
1490         #4

```

```

1491 }
1492 }

```

6.11.4 Conversions

`\__bnvs_convert:nc` `\__bnvs_convert:nc {<definitions>} {<var>}`

Expand and convert `<definitions>` into `<var>`, on return the variable only contains characters with category code other.

```

1493 \BNVS_new:cpn { convert:nc } #1 #2 {
1494   \__bnvs_tl_set:ce { #2 } { \exp_args:Nc \cs_to_str:N { #1 } }
1495 }

```

`\__bnvs_prepare:nnc` `\__bnvs_prepare:nnc {<key>} {<error>} {<var>}`

Used by `\__bnvs_prepare_crl:nc`, `\__bnvs_prepare_sqr:nc` and `\__bnvs_prepare_rnd:nc`.
Implementation detail: local functions `\BNVS:N` and `\BNVS_loop:`.

```

1496 \BNVS_new:cpn { prepare:nnc } #1 #2 #3 {
1497   \cs_set:Npn \BNVS:N ##1 {
1498     \cs_set:Npn \BNVS_loop: {
1499       \exp_args:Nc \regex_replace_all:NnNT { c__bnvs_#1_n_regex } {
1500         \c { BNVS_#1:n } \cB \{ \1 \cE \}
1501       } ##1 {
1502         \BNVS_loop:
1503       }
1504     }
1505     \BNVS_loop:
1506     \exp_args:Nc \regex_match:NVT { c__bnvs_#1_regex } ##1 {
1507       \BNVS_error:n { #2 }
1508     }
1509   }
1510   \BNVS_tl_use:Nc \BNVS:N { #3 }
1511 }

```

`\c__bnvs_crl_n_regex` Used by `\__bnvs_prepare_crl:c` and `\__bnvs_resolve_prepare:nc`.
`\regex_const:Nn \c__bnvs_crl_n_regex { \c0 \{ ( .*? ) \c0 \} }`
(End of definition for \c__bnvs_crl_n_regex.)

`\c__bnvs_crl_regex` Only used by `\__bnvs_prepare_crl:c`.
`\regex_const:Nn \c__bnvs_crl_regex { \c0 \{ | \c0 \} }`
(End of definition for \c__bnvs_crl_regex.)

`\__bnvs_prepare_crl:c` `\__bnvs_prepare_crl:c {<var>}`

Replace any literal `{<...>}` by `\BNVS_crl:n{<...>}`.

```

1514 \BNVS_new:cpn { prepare_crl:c } #1 {
1515   \__bnvs_prepare:nnc { crl } { Unbalanced~\{~or~\ } { #1 }
1516 }

```

`\c__bnvs_sqr_n_regex` Used by `\__bnvs_prepare_sqr:c` and `\__bnvs_resolve_prepare:nc`.
`\regex_const:Nn \c__bnvs_sqr_n_regex { \[ ( [%\]`
`~\[*) \]`
(End of definition for \c__bnvs_sqr_n_regex.)

`\c__bnvs_sqr_regex` Only used by `\__bnvs_prepare_sqr:c`.
`\regex_const:Nn \c__bnvs_sqr_regex { \[|\]`

(End of definition for \c__bnvs_sqr_regex.)

__bnvs_prepare_sqr:c **__bnvs_prepare_sqr:c** {<var>}

Replace any literal [$\langle \dots \rangle$] by \BNVS_sqr:n{<...>}.

```
1520 \BNVS_new:cpn { prepare_sqr:c } #1 {
1521   \__bnvs_prepare:nnc { sqr } { Unbalanced~[~or~] } { #1 }
1522 }
```

\c__bnvs_rnd_n_regex Only used by **__bnvs_prepare_rnd:c**.

```
1523 \regex_const:Nn \c__bnvs_rnd_n_regex { \ ( ( [%]
1524   ~\[ ]*) \) }
```

(End of definition for \c__bnvs_rnd_n_regex.)

\c__bnvs_rnd_regex Only used by **__bnvs_prepare_rnd:c**.

```
1525 \regex_const:Nn \c__bnvs_rnd_regex { \ (| \) }
(End of definition for \c__bnvs_rnd_regex.)
```

__bnvs_prepare_rnd:c **__bnvs_prepare_rnd:c** {<var>}

Replace any literal ($\langle \dots \rangle$) by \BNVS_rnd:n{<...>}.

```
1526 \BNVS_new:cpn { prepare_rnd:c } #1 {
1527   \__bnvs_prepare:nnc { rnd } { Unbalanced~(~or~) } { #1 }
1528 }
```

__bnvs_prepare:nc **__bnvs_prepare:nc** {<definitions>} {<var>}

Prepare the <definitions> into <var>. Expand and convert the <definitions>. Replace any literally material $\langle \dots \rangle$ enclosed between standard delimiters, by one of template \BNVS_crl:n{<...>}, \BNVS_sqr:n{<...>} or \BNVS_rnd:n{<...>}. After that, we are sure that commas are proper delimiters.

```
1529 \BNVS_new:cpn { prepare:nc } #1 #2 {
1530   \__bnvs_convert:nc { #1 } { #2 }
1531   \__bnvs_prepare_crl:c { #2 }
1532   \__bnvs_prepare_sqr:c { #2 }
1533   \__bnvs_prepare_rnd:c { #2 }
1534 }
```

6.11.5 Other utilities

```
1535 \BNVS_new:cpn { warn_until_s_stop:w } #1 \s_stop {
1536   \tl_if_empty:nF { #1 } {
1537     \BNVS_warning:n { Ignored:~#1 }
1538   }
1539 }
1540 \BNVS_new:cpn { scan_until_s_stop:w } {
1541   \peek_meaning:NTF \s_stop {
1542     \use_none:n
1543   } {
1544     \__bnvs_warn_until_s_stop:w
1545   }
1546 }
```

```

__bnvs_peek_branch_until_s_stop:nnnnw  __bnvs_peek_branch_until_s_stop:nnnnw
{<sqr:n>} {<crl:n>} {<non empty:n>} {<empty:>}
{...} \s_stop

```

Branch according to the following token. Each argument, except `<empty:>`, is the body of a function that takes one argument. It catches errors like `foo=[bar]baz`, where `baz` is not expected: no other argument should lay before `\s_stop`.

```

1547 \cs_new:Npn \BNVS_sqr:n { \exp_not:N \BNVS_sqr:n }
1548 \cs_new:Npn \BNVS_crl:n { \exp_not:N \BNVS_crl:n }
1549 \BNVS_new:cpn { peek_branch_until_s_stop:nnnnw } #1 #2 #3 #4 {
1550   \peek_meaning:NTF \BNVS_sqr:n {
1551     \cs_set:Npn \BNVS_peek_branch:n ##1 { #1 }
1552     \cs_set:Npn \BNVS_peek_branch:nn ##1 ##2 {
1553       \BNVS_peek_branch:n { ##2 }
1554       \__bnvs_warn_until_s_stop:w
1555     }
1556     \BNVS_peek_branch:nn
1557   } {
1558     \peek_meaning:NTF \BNVS_crl:n {
1559       \cs_set:Npn \BNVS_peek_branch:n ##1 { #2 }
1560       \cs_set:Npn \BNVS_peek_branch:nn ##1 ##2 {
1561         \BNVS_peek_branch:n { ##2 }
1562         \__bnvs_warn_until_s_stop:w
1563       }
1564       \BNVS_peek_branch:nn
1565     } {
1566       \peek_meaning:NTF \s_stop {
1567         #4
1568         \use_none:n
1569       } {
1570         \cs_set:Npn \BNVS_peek_branch:w ##1 \s_stop {
1571           #3
1572         }
1573         \BNVS_peek_branch:w
1574       }
1575     }
1576   }
1577 }

```

6.11.6 Next index

```

1578 \BNVS_int_new:c { next_index }
1579 \BNVS_new:cpn { next_index:nn } #1 #2 {
1580   \__bnvs_int_incr:c { next_index }
1581   \exp_args:Nnne \__bnvs_is_gset:nnnT { #1 } { #2 } {
1582     = \int_use:N \l__bnvs_next_index_int
1583   } {
1584     \__bnvs_next_index:nn { #1 } { #2 }
1585   }
1586 }

```

```

__bnvs_next_index:nnn  __bnvs_next_index:nnn {<id>} {<path>} {<code:n>}

```

`<code:n>` takes one argument: the next available index.

```

1587 \BNVS_new:cpn { next_index:nnn } #1 #2 #3 {
1588   \BNVS_begin:
1589   \__bnvs_int_zero:c { next_index }
1590   \__bnvs_next_index:nn { #1 } { #2 }
1591   \cs_set:Npn \BNVS:n ##1 {
1592     \BNVS_end:
1593     \__bnvs_int_set:cn { next_index } { ##1 }
1594     #3
1595   }
1596   \BNVS_int_use:Nv \BNVS:n { next_index }
1597 }

```

6.12 Definition

Lower level function names generally contain `_gdef`. We parse definitions and collect material between curly or square brackets. There is a pretreatment before the storage to anticipate the resolution.

<code>__bnvs_gdef:nnn</code> <code>__bnvs_gdef:(vvn nvn)</code>	<code>__bnvs_gdef:nnn {<id>} {<tag>} {<definition>}</code> Called by <code>__bnvs_root_parsed:n</code> and <code>__bnvs_root_parsed:nn</code> . Here is the documentation . The typical example of usage is <code>\Beanoves{<id>}{<tag>}{<definitions>}</code> . If <code><definitions></code> is empty, it clears out the data.
--	---

```

1598 \BNVS_new:cpn { gdef:nnn } #1 #2 #3 {
1599   \__bnvs_peek_branch_until_s_stop:nnnnw {
    To manage <tag>=[<definitions>] call \__bnvs_sqr_gdef:nnn (and possibly itself):
1600     \__bnvs_sqr_gdef:nnn { #1 } { #2 } { ##1 }
1601   } {
    To manage <tag>={<definitions>} call \__bnvs_crl_gdef:nnn (and possibly itself):
1602     \BNVS_begin:
    To manage <tag>={{<definitions>}}:
1603     \__bnvs_tl_set:cn V { ##1 }
1604     \cs_set:Npn \BNVS_loop: {
1605       \BNVS_tl_last_unbraced:nv {
1606         \__bnvs_peek_branch_until_s_stop:nnnnw { } {
1607           \__bnvs_tl_set:cn V { ####1 }
1608           \BNVS_loop:
1609         } {} {}
1610       } V \s_stop
1611     }
1612     \BNVS_loop:
1613     \BNVS_tl_use:nv { \__bnvs_crl_gdef:nnn { #1 } { #2 } } V
1614     \BNVS_end:
1615   } {
    To manage <tag>=<value> call \__bnvs_ginit:nnn:
1616     \__bnvs_ginit:nnn { #1 } { #2 } { ##1 }
1617   } {
    In other cases, call \__bnvs_gclear:nn:
1618     \__bnvs_gclear:nn { #1 } { #2 }
1619   } #3 \s_stop
1620 }

```



```

1621 \BNVS_new:cpn { gdef:nvn } #1 {
1622   \BNVS_tl_use:nv { \__bnvs_gdef:nnn { #1 } }
1623 }
1624 \BNVS_new:cpn { gdef:vvv } {
1625   \BNVS_tl_use:Nvv \__bnvs_gdef:nnn
1626 }
  For <id>!(<path>=<def>) where <def> is the most general one. Workflow: \__bnvs_gclear:nn
  or
  \__bnvs_keyval:ncc
  \__bnvs_gclear:nn or \__bnvs_ginit_sqr:nnn
  \__bnvs_gclear:nn or \__bnvs_gdef:nnn
  \__bnvs_ginit:nnn.

```

6.12.1 Preparation

6.12.2 Material between square brackets

Here is the [documentation](#). The name of the function contains “square” or “sqr”. Workflow:

```

  \__bnvs_sqr_parsed_ITPD:w
  \__bnvs_ginit_sqr:nnn
  \__bnvs_sqr_parsed_ITD:w or \__bnvs_sqr_parsed_ITPD:w
  \__bnvs_ginit_sqr:nnn or \__bnvs_ginit:nnn.
  \__bnvs_if_set:ncccTF \__bnvs_gunset_deep:nn.
  Example: \Beanoves{Id!Tag=[1=A,B,3=C]}.

```

```

\__bnvs_sqr_parsed_ITD:w \__bnvs_sqr_parsed_ITD:w {<id>} {<tag>} {<definition>}

```

For: \Beanoves{<id>!(<path>=[<...>,<definition>,<...>])},

```

1627 \BNVS_new:cpn { sqr_parsed_ITD:w } #1 #2 #3 {
  Find the first available index.
1628   \__bnvs_next_index:nnn { #1 } { #2 } {
1629     \__bnvs_sqr_parsed_ITPD:w { #1 } { #2 } { ##1 } { #3 }
1630   }
1631 }

```

```

\__bnvs_if_index:nTF \__bnvs_if_index:nTF {<what>} {<yes:n>} {<no:>}

```

If <what> is an index, execute <yes:n> otherwise execute <no:>. The <yes:n> argument is the body of a function with one argument: the normalized index built from <what>.

```

1632 \BNVS_new:cpn { if_index:nTF } #1 #2 #3 {
1633   \BNVS_begin:
1634     \__bnvs_match_if_once:NnTF \c__bnvs_A_index_Z_regex { #1 } {
1635       \__bnvs_match_pop:c N
1636       \cs_set:Npn \BNVS:n ##1 {
1637         \BNVS_end:
1638         #2
1639       }
1640       \BNVS_tl_use:Nv \BNVS:n N
1641     } {
1642       \cs_set:Npn \BNVS:n ##1 {
1643         \BNVS_end:

```

```

1644     #3
1645   }
1646   \BNVS:n { #1 }
1647 }
1648 }

```

_bnvs_sqr_parsed_ITPD:w _bnvs_sqr_parsed_ITPD:w {<id> } {<tag> } {<key> } {<definition> }

For: <tag>=[<...>,<key>=<definition>,<...>].

```

1649 \BNVS_new:cpn { sqr_parsed_ITPD:w } #1 #2 #3 #4 {
1650   \_bnvs_if_index:nTF { #3 } {
1651     \_bnvs_is_gset_and_provide:nnnF { #1 } { #2 } { = ##1 } {
1652       \_bnvs_ginit_ITPD:w { #1 } { #2 } { ##1 } { #4 }
1653     }
1654   } {
1655     \_bnvs_next_index:nnn { #1 } { #2 } {
1656       \_bnvs_ginit_ITPD:w { #1 } { #2 } { ##1 } { #4 }
1657     }
1658   }
1659 }

```

_bnvs_ginit_sqr:nnn _bnvs_ginit_sqr:nnn {<id> } {<path> } {<definitions> }

On execution, the root t1 variable is set and not empty. Called by **(IN PROGRESS)** to parse what is inside square brackets:

- \Beanoves{<path>=[<definitions>]]},
- \Beanoves{<id>!<path>=[<definitions>]]}.

Calls _bnvs_sqr_parsed_ITD:w and _bnvs_sqr_parsed_ITPD:w. Supports provide mode by calling _bnvs_is_gset_and_provide:nnnTF.

```

1660 \BNVS_new:cpn { ginit_sqr:nnn } #1 #2 #3 {
  Will remove what is related to the tag when not in provide mode.
1661   \tl_if_empty:nTF { #2 } {
1662     \BNVS_error:n { Unexpected~list~at~top~level. }
1663   } {
1664     \tl_if_empty:nF { #3 } {
1665       \_bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
1666         \_bnvs_gunset_deep:nn { #1 } { #2 }
1667         \BNVS_begin:
1668         \keyval_parse:nnn
1669         { \_bnvs_sqr_parsed_ITD:w { #1 } { #2 } }
1670         { \_bnvs_sqr_parsed_ITPD:w { #1 } { #2 } }
1671         { #3 }
1672         \BNVS_end:
1673       }
1674     }
1675   }
1676 }
1677 \BNVS_new:cpn { ginit_sqr:nvn } #1 {
1678   \BNVS_tl_use:nv { \_bnvs_ginit_sqr:nnn { #1 } }
1679 }

```

```

1680 \BNVS_new:cpn { ginit_sqr:vvv } {
1681   \BNVS_tl_use:Nvv \_bnvs_ginit_sqr:nnn
1682 }
1683 \BNVS_new:cpn { ginit_sqr_I:nn } {
1684   \BNVS_tl_use:cv { ginit_sqr:nnn } I
1685 }
1686 \BNVS_new:cpn { ginit_sqr_I:vn } {
1687   \BNVS_tl_use:cv { ginit_sqr_I:nn }
1688 }

```

6.12.3 Top level

_bnvs_root_parsed:n _bnvs_root_parsed:n {<key>}

Called from _bnvs_root_keyval:nc. For \Beanoves{<key>} instruction. At the root level, the <key> is either <path> or <id>!*<path>*, it is not a <subpath>. Calls _bnvs_if_IT:nTF to parse the <key>, _bnvs_ginit:nnn to define the value.

```

1689 \BNVS_new:cpn { root_parsed:n } #1 {
1690   \_bnvs_if_IT:nTF { #1 } {
1691     \_bnvs_gdef:vvv IT 1
1692   } {
1693     \BNVS_error:n { Unsupported-#1 }
1694   }
1695 }

```

_bnvs_root_parsed:nn _bnvs_root_parsed:nn {<key>} {<value>}

Called from _bnvs_root_keyval:nc. For \Beanoves{<key>=<definition>} instruction. At the root level, the <key> is either <path> or <id>!*<path>*, it is not a <subpath>. Calls _bnvs_if_IT:nTF to parse the <key> and _bnvs_gdef:nnn to set the definition.

```

1696 \BNVS_new:cpn { root_parsed:nn } #1 #2 {
1697   \_bnvs_if_IT:nTF { #1 } {
1698     \_bnvs_gdef:vvv IT { #2 }
1699   } {
1700     \BNVS_error:n { Unsupported-#1 }
1701   }
1702 }

```

_bnvs_root_keyval:nc _bnvs_root_keyval:nc {<definitions>} {<aux>}

Is called by \BeanovesReset and by \Beanoves through _bnvs_root_keyval:NNn, <definitions> directly comes from the user input. <aux> is an auxiliary variable only used within the body of the function, its content on input and output must be ignored whereas J may be updated.

1. Calls _bnvs_prepare:nc to prepare <definitions> into the <aux> variable, which will contain only characters (with category code other).
2. Parse with keyval based on \keyval_parsed:nn helped by _bnvs_root_parsed:n and _bnvs_root_parsed:nn.

```

1703 \BNVS_new:cpn { root_keyval:nc } #1 #2 {
1704   \__bnvs_prepare:nc { #1 } { #2 }
1705   \BNVS_tl_use:nv {
1706     \keyval_parse:NNn \__bnvs_root_parsed:n \__bnvs_root_parsed:nn
1707   } { #2 }
1708 }

```

__bnvs_root_keyval:NNn **__bnvs_root_keyval:NNn** *<is at top> <on provide mode> {<definitions>}*

Called by **\Beanoves**. Calls **__bnvs_root_keyval:nc** after some setup. The a tl auxiliary variable is used and, on return, the J variable may have been updated.

```

1709 \BNVS_new:cpn { root_keyval:NNn } #1 #2 #3 {
1710   \bool_if:NT #1 {
❧ At the document level, clear everything.
1711     \__bnvs_gclear:
1712   }
1713   \BNVS_begin:
1714   \bool_if:NTF #2 {
1715     \__bnvs_provide_ON:
1716   } {
1717     \__bnvs_provide_OFF:
1718   }
1719   \__bnvs_int_zero:c { i }
1720   \__bnvs_tl_set:cn a { #3 }
1721   \bool_if:NT #1 {
❧ At the document level, also use the global definitions.
1722     \seq_if_empty:NF \g__bnvs_def_seq {
1723       \__bnvs_tl_put_left:cx a {
1724         \seq_use:Nn \g__bnvs_def_seq , ,
1725       }
1726     }
1727   }
1728   \BNVS_tl_use:Nv \__bnvs_root_keyval:nc a a
  This is a list, possibly at the top
1729   \BNVS_end_tl_set:cv J J
1730 }

```

\Beanoves **\Beanoves** *{<key-value list>}*

The keys are the slide overlay references. When no value is provided, it defaults to 1.

On the contrary, *<key-value>* items are parsed by **__bnvs_root_keyval:NNn**.

```

1731 \NewDocumentCommand \Beanoves { sm } {
1732   \__bnvs_set_false:c { reset }
1733   \__bnvs_set_false:c { reset_all }
1734   \__bnvs_set_false:c { only }
1735   \tl_if_empty:NTF \@currentenv {
❧ In the preamble, record the definitions globally for later use. No expansion yet.
1736     \seq_gput_right:Nn \g__bnvs_def_seq { #2 }
1737   } {
1738     \tl_if_eq:NnTF \@currentenv { document } {
1739       \IfBooleanTF {#1} {
1740         \__bnvs_root_keyval:NNn \c_true_bool \c_true_bool
1741       } {

```

```

1742     \_bnvs_root_keyval:NNn \c_true_bool \c_false_bool
1743   }
1744   } {
1745     \IfBooleanTF {#1} {
1746       \_bnvs_root_keyval:NNn \c_false_bool \c_true_bool
1747     } {
1748       \_bnvs_root_keyval:NNn \c_false_bool \c_false_bool
1749     }
1750   } { #2 }
1751   \ignorespaces
1752 }
1753 }

```

If we use the frame `beanoves` option, we can provide default values to the various overlay set names through the use of `\Beanoves*`.

```

1754 \define@key{beamerframe}{beanoves}{\Beanoves*{#1}}

```

6.13 Scanning named overlay specifications

Patch some beamer commands to support `?(...)` instructions in overlay specifications.

<code>_bnvs_@frame</code> <code>_bnvs_@masterdecode</code>	<code>_bnvs_@frame {<overlay specification>}</code> <code>_bnvs_@masterdecode {<overlay specification>}</code>
---	---

Preprocess `<overlay specification>` before beamer reads it. Both call `\_bnvs\_resolve:nc`.
`\l\_bnvs\_ans\_tl` Storage for the translated overlay specification, `<...>` instructions are replaced by their static counterparts.
(End of definition for `\l\_bnvs\_ans\_tl`.)

Save the original macros `\beamer@frame` and `\beamer@masterdecode` then override them to properly preprocess the argument. We start by defining the overloads.

```

1755 \makeatletter
1756 \cs_set:Npn \_bnvs\_@frame < #1 > {
1757   \BNVS_begin:
1758   \_bnvs\_tl\_clear:c { ans }
1759   \_bnvs\_resolve:nc { #1 } { ans }
1760   \BNVS_set:cpn { :n } ##1 { \BNVS_end: \BNVS_saved@frame < ##1 > }
1761   \BNVS\_tl\_use:cv { :n } { ans }
1762 }
1763 \cs_set:Npn \_bnvs\_@masterdecode #1 {
1764   \BNVS_begin:
1765   \_bnvs\_tl\_clear:c { ans }
1766   \_bnvs\_resolve:nc { #1 } { ans }
1767   \BNVS\_tl\_use:nv {
1768     \BNVS_end:
1769     \BNVS_saved@masterdecode
1770   } { ans }
1771 }
1772 \cs_new:Npn \BeanovesOff {
1773   \cs_set_eq:NN \beamer@frame \BNVS_saved@frame
1774   \cs_set_eq:NN \beamer@masterdecode \BNVS_saved@masterdecode
1775 }
1776 \cs_new:Npn \BeanovesOn {
1777   \cs_set_eq:NN \beamer@frame \_bnvs\_@frame
1778   \cs_set_eq:NN \beamer@masterdecode \_bnvs\_@masterdecode
1779 }

```

```

1780 \AddToHook{begindocument/before}{
1781   \cs_if_exist:NTF \beamer@frame {
1782     \cs_set_eq:NN \BNVS_saved@frame \beamer@frame
1783     \cs_set_eq:NN \BNVS_saved@masterdecode \beamer@masterdecode
1784   } {
1785     \cs_set:Npn \BNVS_saved@frame < #1 > {
1786       \BNVS_error:n {Missing~package~beamer}
1787     }
1788     \cs_set:Npn \BNVS_saved@masterdecode < #1 > {
1789       \BNVS_error:n {Missing~package~beamer}
1790     }
1791   }
1792   \BeanovesOn
1793 }
1794 \makeatother

```

6.14 Resolution

Resolution means to transform $\langle \langle \text{queries} \rangle \rangle$ and alike into static `beamer` range expressions. This is a quite complex task because we allow at the same time algebraic expressions and `beamer` range expressions which seem like differences but are not algebraic expressions by themselves. Function names generally contain “resolve”.

Each individual query is scanned multiple times, changing parts on each step to finally obtain only raw integers (along with management markers). Overall sketch: `\__bnvs_resolve:nc` is the top level resolution function. It is called by `\__bnvs_@frame`, `\__bnvs_@masterdecode` or `\BeanovesResolve` and for the resolution of embedded subqueries $\langle \langle \text{queries} \rangle \rangle$ or $!:\langle \langle \text{queries} \rangle \rangle$ as well. The main functions used for resolution are

❗ `\__bnvs_resolve_query:nc`, called first, the input is a raw query, which is not a clist object, on return the answer is **(IN PROGRESS)**

- `\BNVS_V:w{\langle v \rangle}`, for a single integer literal value $\langle v \rangle$,
- `\BNVS_AZ:w{\langle a \rangle}{\langle z \rangle}`, for a single range of integers from literal integer $\langle a \rangle$ to literal integer $\langle z \rangle$, or from literal integer $\langle a \rangle$ if $\langle z \rangle$ is empty,
- a comma separated list of the above

The result may be used as a single integer value as well as a `beamer` list of ranges. Depending on the context, the functions will be defined appropriately.

❗ `\__bnvs_resolve_prepare:nc`, called first by `\__bnvs_resolve_query:nc`, on return the prepared material consists of

- `\BNVS_paren:n{\langle \dots \rangle}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{\langle b \rangle}{\langle I \rangle}{\langle K \rangle}{\langle D \rangle}{\langle P \rangle}{\langle T \rangle}{\langle N \rangle}{\langle U \rangle}{\langle a \rangle}`, for specifications
- `\BNVS_Ua:w{\langle U \rangle}{\langle a \rangle}` for static attributes after a “.” character and following ++,
- `\BNVS_n:n{\langle \dots \rangle}` for an integer expression between square brackets,
- `\BNVS_s:n{\langle \dots \rangle}` for a general subquery inside $\langle \langle \dots \rangle \rangle$,

- `\BNVS_assign:n{⟨/+/?⟩}`, for =, += or ?=,
 - `\BNVS_two_colons:` and `\BNVS_colon:` are range delimiters
 - `\BNVS_comma:` for commas
 - `\BNVS_raw_to_stop:w⟨raw text⟩\s_stop` for everything else.
- ¶ `\__bnvs_resolve_subqueries:c`, after resolution of subqueries the material consists of
- `\BNVS_bIPTNUa:w{⟨b⟩}{⟨I⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}`, for specifications
 - `\BNVS_Ua:w{.⟨attributes⟩}{⟨number of ++⟩}`,
 - `\BNVS_N:w{⟨...⟩}` for a single static resolved integer,
 - `\BNVS_S:w{⟨...⟩}` containing output similar to that of `\__bnvs_resolve_query:nc`,
 - `\BNVS_assign:n{⟨/+/?⟩}`, for =, += or ?=,
 - `\BNVS_two_colons:` and `\BNVS_colon:`, range delimiters.
 - `\BNVS_comma:` for commas
 - `\BNVS_raw:n{⟨raw text⟩}` for everything else.
- ¶ `\__bnvs_resolve_check:c` after resolution the material consists of
- `\BNVS_bIPTNUa:w{⟨b⟩}{⟨I⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}`, for specifications
 - `\BNVS_N:w{⟨...⟩}` for a single static resolved integer,
 - `\BNVS_S:w{⟨...⟩}` for a comma separated list of static resolved $\langle A \rangle$, $\langle A \rangle$: and $\langle A \rangle:\langle Z \rangle$ ranges, $\langle A \rangle$ and $\langle Z \rangle$ being unsigned integer decimal denotations,
 - `\BNVS_assign:n{⟨/+/?⟩}`, for =, += or ?=,
 - `\BNVS_two_colons:` and `\BNVS_colon:`, range delimiters.
 - `\BNVS_comma:` for commas
 - `\BNVS_raw:n{⟨raw text⟩}` for everything else.
- ¶ `\__bnvs_resolve_rhs:c` after resolution of assignments the material consists of
(IN PROGRESS)
- `\BNVS_V:w{⟨v⟩}`, for a single integer literal value $\langle v \rangle$,
 - `\BNVS_AZ:w{⟨a⟩}{⟨z⟩}`, for a single range of integers from literal integer $\langle a \rangle$ to literal integer $\langle z \rangle$, or from literal integer $\langle a \rangle$ if $\langle z \rangle$ is empty,
 - a comma separated list of the above
- ¶ `\__bnvs_resolve_assign:nc` after resolution of assignments the material consists of
- `\BNVS_bIPTNUa:w{⟨b⟩}{⟨I⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}`, for specifications
 - `\BNVS_N:w{⟨...⟩}` for a single static resolved integer,
 - `\BNVS_S:w{⟨...⟩}` for a comma separated list of static resolved $\langle A \rangle$, $\langle A \rangle$: and $\langle A \rangle:\langle Z \rangle$ ranges, $\langle A \rangle$ being positive and $\langle Z \rangle$ being nonnegative,
 - `\BNVS_two_colons:` and `\BNVS_colon:`, range delimiters.
 - `\BNVS_comma:` for commas

– `\BNVS_raw:n{<raw text>}` for everything else.

`\_bnvs_assign:Nwnnc`

These functions define the function in the input and run the input.

The top level queries are a comma separated list of individual queries as augmented range specifications. Each individual query is temporarily stored in its own variable, then commands are inserted as markup. This variable is then executed to feed the top level resolution variable.

The augmentations in individual queries can take different forms. At the top level we have only `?(<queries>)` or `!:(<queries>)` and inside we may have

- embedded `?(<queries>)` or `!:(<queries>)`,
- colon delimited ranges,
- various assignments (standard, with increment and as provider),
- references with possible square brackets specification,
- pre and post increment.

The right hand side expression in assignments may contain commas. In that case there can be a conflict with top level commas. To avoid such ambiguity, one can put rounded brackets around the critical part.

`\c__bnvs_bIKDPN_regex` Used by `\_bnvs_resolve_prepare:nc`.

```

1795 \regex_const:Nn \c__bnvs_bIKDPN_regex {
1796   ( (? : \+ \+ ) * )
1797   (? : ( \ur{c__bnvs_S_regex} ) ? ( ! ) | ( \. ) ) ?
1798   ( \ur{c__bnvs_S_regex} ( ? : \. \ur{c__bnvs_S_regex} ) * )
1799   ( ? : \. \s* ( [ - + \s ] * \s* \d+ ) ) ?
1800   \s*
1801 }
(End of definition for \c__bnvs_bIKDPN_regex.)

```

`\c__bnvs_bIKDPTNUa_regex` Used by `\_bnvs_resolve_prepare:nc`.

- `<b>`: an optional collection of ++ prefixes,
- `<I>`: an optional `<id>`
- `<K>`: followed by a “!” or
- `<D>`: an optional “.”,
- `<P>`: a dotted path,
- `<T>`: optional dotted `<attributes>`,
- `<N>`: a litteral optionally signed integer,
- `<U>`: other optional dotted `<attributes>`,
- `<a>`: an optional collection of ++ suffixes.


```

1802 \regex_const:Nn \c__bnvs_bIKDPTNUa_regex {
1803   ( (? : \+ \+ ) * )
1804   (? : ( \ur{c__bnvs_S_regex} ) ? ( ! ) | ( \. ) ) ?
1805   ( \ur{c__bnvs_S_regex} ( ? : \. \ur{c__bnvs_S_regex} ) * )
1806   ( (? : \. \ur{c__bnvs_U_regex} ) * )
1807   (? : \. \s* ( [-+ \s] * \s* \d+ ) ) ?
1808   ( (? : \. \ur{c__bnvs_U_regex} ) * )
1809   ( (? : \+ \+ ) * )
1810   \s*
1811 }

```

(End of definition for `\c__bnvs_bIKDPTNUa_regex`.)

`\c__bnvs_UaU_regex` Used by `\__bnvs_resolve_prepare:nc`.

`<U>`: optional dotted `<attributes>` and

`<a>`: post increment,

`<U>`: or standalone optional dotted `<attributes>`.

```

1812 \regex_const:Nn \c__bnvs_UaU_regex {
1813   (? : ( (? : \. \ur{c__bnvs_U_regex} ) * ) ( (? : \+ \+ ) + )
1814   | ( (? : \. \ur{c__bnvs_U_regex} ) + ) )
1815   \s*
1816 }

```

(End of definition for `\c__bnvs_UaU_regex`.)

`\__bnvs_resolve_prepare:nc` `\__bnvs_resolve_prepare:nc {<queries>} {<var>}`

Called by `\__bnvs_resolve_query:nc` and by `\__bnvs_resolve_queries:nc`. Prepare the `<queries>` into `<var>`. Inside `<queries>`, ranges are denoted with a colon syntax.

On return, every material has been tagged between commands, as a function or a command argument. More precisely `<var>` consists of

- `\BNVS_paren:n{<...>}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{<b>}{<I>}{<K>}{<D>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_Ua:w{<U>}{<a>}` for static attributes after a “.” character and following ++,
- `\BNVS_n:n{<...>}` for an integer expression between square brackets,
- `\BNVS_s:n{<...>}` for a general subquery inside `?(<...>)`,
- `\BNVS_assign:n{</+/?>}`, for =, += or ?=,
- `\BNVS_two_colons:` and `\BNVS_colon:` are range delimiters
- `\BNVS_comma:` for commas
- `\BNVS_raw_to_stop:w{raw text}\s_stop` for everything else.

```

1817 \BNVS_new:cpn { resolve_prepare:nc } #1 #2 {

```

- ¶ One shot function, subsequent calls within the same group are just restricted to the raw assignement of `<queries>` to `<var>`, because the `<queries>` argument here is some part of a more general `<queries>` that has already been prepared.

```

1818 \BNVS_set:cpn { resolve_prepare:nc } ##1 ##2 {
1819   \__bnvs_tl_set:cn { ##2 } { ##1 }
1820 }

```

❗ Turn the queries into a string by expanding macros, #2 will contain only characters.

```

1821 \tl_if_empty:nTF { #1 } {
1822   \__bnvs_tl_clear:c { #2 }
1823 } {
1824   \__bnvs_tl_set:ce { #2 } { \exp_args:Nc \cs_to_str:N { #1 } }
1825 }

```

❗ Enclose the material between commas, there is no grouping yet so this is not a comma separated list. The leading and trailing marks will be inserted at the end of this function body.

```

1826 \BNVS_tl_use:Nc \tl_replace_all:Nnn { #2 }
1827 { , } { \s_stop \BNVS_comma: \BNVS_raw_to_stop:w}

```

❗ Replace “(⟨...⟩)” with

```

\s_stop
\BNVS_paren:n{\BNVS_raw_to_stop:w⟨...⟩\s_stop}
\BNVS_raw_to_stop:w.

```

Standard delimiters are expected to be properly balanced, no error recovery policy for that matter. Category codes now come back into play.

```

1828 \cs_set:Npn \BNVS_loop: {
1829   \BNVS_tl_use:nc { \regex_replace_all:NnNT \c__bnvs_rnd_n_regex {
1830     \c { s_stop }
1831     \c { BNVS_paren:n } \cB \{
1832     \c { BNVS_raw_to_stop:w } \1 \c { s_stop }
1833     \cE \}
1834     \c { BNVS_raw_to_stop:w }
1835   } } { #2 } \BNVS_loop:
1836 }
1837 \BNVS_loop:

```

❗ Replace “[⟨...⟩]” with

```

\s_stop
\BNVS_v:n{\BNVS_raw_to_stop:w⟨...⟩\s_stop}
\BNVS_raw_to_stop:w.

```

The content will be resolved as a static signed integer and will be used as an index.

```

1838 \cs_set:Npn \BNVS_loop: {
1839   \BNVS_tl_use:nc { \regex_replace_all:NnNT \c__bnvs_sqr_n_regex {
1840     \c { s_stop } \c { BNVS_n:n } \cB \{
1841     \c { BNVS_raw_to_stop:w } \1 \c { s_stop }
1842     \cE \} \c { BNVS_raw_to_stop:w }
1843   } } { #2 } \BNVS_loop:
1844 }
1845 \BNVS_loop:

```

❗ Also replace references, increments, assignments and colons.

```

1846 \BNVS_tl_use:nc { \regex_replace_case_all:nN {

```

Basically, “?(⟨...⟩)” is replaced by “\BNVS_s:n{⟨...⟩}” that will be evaluated as a subquery for a list of values and ranges.

```

1847 { (?: \?!: ) \c { s_stop } \c { BNVS_paren:n } } {
1848   \c { s_stop } \c { BNVS_s:n }
1849 }

```

```

1850 { \c__bnvs_bIKDPTNUa_regex } {
1851   \c { s_stop }
1852   \c { BNVS_bIKDPTNUa:w }
1853     { \1 } { \2 } { \3 } { \4 } { \5 } { \6 } { \7 } { \8 } { \9 }
1854   \c { BNVS_raw_to_stop:w }
1855 }
  Post increment operators and attributes are collected
1856 { \c__bnvs_UaU_regex } {
1857   \c { s_stop }
1858   \c { BNVS_Ua:w } \cB \{ \1\3 \cE \} \cB \{ \2 \cE \}
1859   \c { BNVS_raw_to_stop:w }
1860 }
  Assignment operators together with their optional modifier are collected
1861 { \s* ([+?])?= \s* } {
1862   \c { s_stop }
1863   \c { BNVS_assign:n } \cB \{ \1 \cE \}
1864   \c { BNVS_raw_to_stop:w }
1865 }
  Colon range delimiters:
1866 { \s* ::+ \s* } {
1867   \c { s_stop }
1868   \c { BNVS_two_colons: }
1869   \c { BNVS_raw_to_stop:w }
1870 }
1871 { \s* : \s* } {
1872   \c { s_stop }
1873   \c { BNVS_colon: }
1874   \c { BNVS_raw_to_stop:w }
1875 }
1876 } } { #2 }
1877 \__bnvs_tl_put_left:cn { #2 } { \BNVS_raw_to_stop:w }
1878 \__bnvs_tl_put_right:cn { #2 } { \s_stop }
1879 }

```

```

\__bnvs_resolve_queries:nc \__bnvs_resolve_queries:nc {<queries>} {<ans>}

```

One of the top level resolution entry point. May be called recursively. For each individual `<query>` of the `<queries>` clist, it replaces in the `<queries>` all the `?(<...>)` query instructions as well as `[<...>]` index references with their static counterpart then it evaluates the instructions to obtain a single value or a set of references. Ranges are in colon (`:`) syntax. The function `\__bnvs_resolve_query:nc` is called for each individual query in the `<queries>` clist. The `Q seq` variable stores the specifications whereas the `Q tl` variable stores the result. The implementation is not highly efficient in the sense that queries are scanned multiple times, but it does not cost that much as long as queries are reasonably short.

```

1880 \BNVS_new:cpn { resolve_queries:nc } #1 #2 {
1881   \BNVS_begin:
1882   \__bnvs_tl_clear:c Q
1883   \__bnvs_seq_clear:c Q
1884   \cs_set:Npn \BNVS_raw:n ##1 {
1885     \tl_if_blank:nF { ##1 } {
1886       \__bnvs_seq_put_right:cn Q { ##1 }
1887     }

```

```

1888 }
1889 \cs_set:Npn \BNVS_set_S:n ##1 {
1890   \__bnvs_tl_if_empty:cT Q {
1891     \__bnvs_tl_set:cn Q { \BNVS_S:w }
1892     \cs_set:Npn \BNVS_S:w #####1 {
1893       \__bnvs_seq_put_right:cn Q { #####1 }
1894     }
1895     \cs_set_eq:NN \BNVS_V:w \BNVS_S:w
1896   }
1897   \__bnvs_seq_put_right:cn Q { ##1 }
1898 }
1899 \cs_set_eq:NN \BNVS_S:w \BNVS_set_S:n
1900 \cs_set:Npn \BNVS_V:w ##1 {
1901   \__bnvs_seq_put_right:cn Q { ##1 }
1902   \cs_set_eq:NN \BNVS_V:w \BNVS_set_S:n
1903 }
1904 \clist_map_inline:nn { #1 } {
1905   \__bnvs_resolve_query:nc { ##1 } { #2 }
1906   \BNVS_tl_use:Nv \use:n { #2 }
1907 }
1908 \__bnvs_tl_if_empty:cT Q {
1909   \__bnvs_tl_set:cn Q { \BNVS_V:w }
1910 }
1911 \exp_args:Nne \__bnvs_tl_put_right:cn Q {
1912   { { \__bnvs_seq_use:cn Q { , } } }
1913 }
1914 \BNVS_end_tl_set:cv { #2 } Q
1915 }

```

```

\__bnvs_resolve_argument:c \__bnvs_resolve_argument:c {<query var>}
\__bnvs_resolve_argument:nc \__bnvs_resolve_argument:nc {<argument>} {<var>}

```

Called by `\__bnvs_resolve_subqueries:nc`.

On input, `<query>` or `<var>` consists of

- `\BNVS_paren:n{<...>}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{<b>}{<I>}{<K>}{<D>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications
- `\BNVS_Ua:w{<U>}{<a>}` for static attributes after a “.” character and following ++,
- `\BNVS_n:n{<...>}` for an integer expression between square brackets,
- `\BNVS_s:n{<...>}` for a general subquery inside `?(<...>)`,
- `\BNVS_assign:n{</+/?>}`, for =, += or ?=,
- `\BNVS_two_colons:` and `\BNVS_colon:` are range delimiters
- `\BNVS_comma:` for commas
- `\BNVS_raw_to_stop:w{raw text}\s_stop` for everything else.

and on output, `<var>` consists of

- `\BNVS_bIPTNUa:w{<b>}{<I>}{<P>}{<T>}{<N>}{<U>}{<a>}`, for specifications

- `\BNVS_N:w{⟨...⟩}` for a single static resolved integer,
- `\BNVS_S:w{⟨...⟩}` for a comma separated list of static resolved $\langle A \rangle$, $\langle A \rangle:$ and $\langle A \rangle:\langle Z \rangle$ ranges, $\langle A \rangle$ being positive and $\langle Z \rangle$ being nonnegative,
- `\BNVS_two_colons:` and `\BNVS_colon:`, range delimiters.
- `\BNVS_comma:` for commas
- `\BNVS_raw:n{⟨raw text⟩}` for everything else.

The $\langle query \ var \rangle$ is executed after the commands it contains are defined on the run. use `\__bnvs_resolve_query:nc`. Run inside a group.

```

1916 \BNVS_new:cpn { resolve_argument:nc } #1 #2 {
1917   \BNVS_begin:
1918   \__bnvs_resolve_subqueries:nc { ##1 } A
1919   \__bnvs_resolve_check:c A
1920   \__bnvs_resolve_assign:c A
1921   \BNVS_end:
1922 }

```

Raw text.

```

\__bnvs_resolve_subqueries:c \__bnvs_resolve_subqueries:c {⟨query var⟩}
\__bnvs_resolve_subqueries:nc \__bnvs_resolve_subqueries:nc {⟨query⟩} {⟨var⟩}

```

Called after `\__bnvs_resolve_prepare:nc`.

On input, $\langle query \rangle$ or $\langle var \rangle$ consists of

- `\BNVS_paren:n{⟨...⟩}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{⟨b⟩}{⟨I⟩}{⟨K⟩}{⟨D⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}`, for specifications
- `\BNVS_Ua:w{⟨U⟩}{⟨a⟩}` for static attributes after a “.” character and following ++,
- `\BNVS_n:n{⟨...⟩}` for an integer expression between square brackets,
- `\BNVS_s:n{⟨...⟩}` for a general subquery inside $?(⟨...⟩)$,
- `\BNVS_assign:n{⟨/+/?⟩}`, for =, += or ?=,
- `\BNVS_two_colons:` and `\BNVS_colon:` are range delimiters
- `\BNVS_comma:` for commas
- `\BNVS_raw_to_stop:w{⟨raw text⟩}\s_stop` for everything else.

and on output, $\langle var \rangle$ consists of

- `\BNVS_bIPTNUa:w{⟨b⟩}{⟨I⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}`, for specifications
- `\BNVS_Ua:w{⟨.attributes⟩}{⟨number of ++⟩}`,
- `\BNVS_N:w{⟨...⟩}` for a single static resolved integer,
- `\BNVS_S:w{⟨...⟩}` containing output similar to that of `\__bnvs_resolve_query:nc`,
- `\BNVS_assign:n{⟨/+/?⟩}`, for =, += or ?=,